Credit Risk and Loan Valuation Modeling: Ryan Gomberg

Abstract

Credit risk modeling is imperative in financial and insurance programs, for it directly influences pricing and risk management decisions. While necessary in underwriting, estimating borrower default risk is critical in understanding the sensitivity and amortizatrion of loan portfolios under dynamic economic conditions.

This project aims to investigate such practices through an integrated credit risk and loan valuation model, combining machine learning techniques based on default probability estimation with survival modeling and fixed-income valuation approaches. Through the construction of multiple classification models, a gradient boosted tree model accomplished substantially strong separability in a borrower's probability of defaulting with an AUC of 97.93%, and establishing strong predictive power with F1-score of 96% (default) and 85% (paid-on-time) with a 77.78%/22.22% class imbalance. The results were then translated into a continuous-time hazard system to approximate survival curves, producing survival probabilities that are incorporated in measuring expected losses for outstanding loan balances, loss given default, and discounting.

Moreover, the project offers a glimpse into loan value sensitivity when subjected to perturbations in interest rate using exact valuation and a first-order Modified Duration approximation. The resulting loan value spectrum over interest rate changes reveals the interaction between credit risk and interest rate risk in loan amortization.

The final model offers a continuous and transformative methodology in credit loan valuation, incorporating statistical learning, actuarial analysis, and fixed-income techniques into a consolidated mathematical model.

Contents

1 Introduction to the Dataset, Objectives **3**

2 Preprocessing **5**

3 Exploratory Data Analysis **10**

4 Models & Methodology **17**

4.1 Logistic Regression . . . 18

4.1.1 Base Model (All Features) . . . 18

4.1.2 Modified Model (Excluding Age & Employment Experience) . . . 18

4.1.3 L1-Regularization Model . . . 20

4.2 Linear Discriminant Analysis . . . 26

4.3 Ensemble Methods . . . 29

4.3.1 Random Forest . . . 29

4.3.2 Boosting . . . 32

4.4 Summary . . . 36

4.5 Conclusion . . . 36

5 Amortization Modeling **37**

6 Default Risk Measurement **44**

7 Expected Loss Modeling **48**

8 Sensitivity **51**

9 Conclusion **54**

10 References **55**

1 Introduction to the Dataset, Objectives

The following results use the bank_loan_data set from Kaggle (https://www.kaggle.com/datasets/udaymalviya/bank-loan-data). The dataset contains 45,000 records of loan borrowers with features attributed to personal demographics, financial status, and loan information.

- Personal demographics: Age, gender, education (highest degree earned), annual income, type of home ownership, years of employment experienced
- Loan properties: Amount requested, intent or purpose, interest rate (%), ratio of loan amount to income
- Credit and Loan History: Length of applicant's credit history (years), applicant's credit score, and whether the applicant has defaulted previously (Yes or no).

The target variable, or the response variable to be predicted, is whether the applicant will successfully pay off the loan or default. Because this is a binary classifier, the dataset delegates the applicant's loan status as 1 = repaid and 0 = defaulted.

In summary, the analysis to be desired from this dataset include:

1. Feature v. Feature and Feature v. Respone interactions
2. Feature counts
3. Binary Classification Model Optimization
4. Loan Amortization Scheduling
5. Default Risk Modeling and Expected Loss Modeling
6. Sensitivity

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import preprocessing

import warnings
warnings.filterwarnings("ignore")

plt.rcParams["font.family"] = "Times New Roman"

df = pd.read_csv('loan_data.csv')
df.head(15)
```

[1]:
```
    person_age person_gender person_education  person_income  person_emp_exp  \
0         22.0        female           Master        71948.0               0
1         21.0        female      High School        12282.0               0
2         25.0        female      High School        12438.0               3
```

3	23.0	female	Bachelor	79753.0	0
4	24.0	male	Master	66135.0	1
5	21.0	female	High School	12951.0	0
6	26.0	female	Bachelor	93471.0	1
7	24.0	female	High School	95550.0	5
8	24.0	female	Associate	100684.0	3
9	21.0	female	High School	12739.0	0
10	22.0	female	High School	102985.0	0
11	21.0	female	Associate	13113.0	0
12	23.0	male	Bachelor	114860.0	3
13	26.0	male	Master	130713.0	0
14	23.0	female	Associate	138998.0	0

	person_home_ownership	loan_amnt	loan_intent	loan_int_rate	\
0	RENT	35000.0	PERSONAL	16.02	
1	OWN	1000.0	EDUCATION	11.14	
2	MORTGAGE	5500.0	MEDICAL	12.87	
3	RENT	35000.0	MEDICAL	15.23	
4	RENT	35000.0	MEDICAL	14.27	
5	OWN	2500.0	VENTURE	7.14	
6	RENT	35000.0	EDUCATION	12.42	
7	RENT	35000.0	MEDICAL	11.11	
8	RENT	35000.0	PERSONAL	8.90	
9	OWN	1600.0	VENTURE	14.74	
10	RENT	35000.0	VENTURE	10.37	
11	OWN	4500.0	HOMEIMPROVEMENT	8.63	
12	RENT	35000.0	VENTURE	7.90	
13	RENT	35000.0	EDUCATION	18.39	
14	RENT	35000.0	EDUCATION	7.90	

	loan_percent_income	cb_person_cred_hist_length	credit_score	\
0	0.49	3.0	561	
1	0.08	2.0	504	
2	0.44	3.0	635	
3	0.44	2.0	675	
4	0.53	4.0	586	
5	0.19	2.0	532	
6	0.37	3.0	701	
7	0.37	4.0	585	
8	0.35	2.0	544	
9	0.13	3.0	640	
10	0.34	4.0	621	
11	0.34	2.0	651	
12	0.30	2.0	573	
13	0.27	4.0	708	
14	0.25	4.0	583	

```
   previous_loan_defaults_on_file  loan_status
0                              No            1
1                             Yes            0
2                              No            1
3                              No            1
4                              No            1
5                              No            1
6                              No            1
7                              No            1
8                              No            1
9                              No            1
10                             No            1
11                             No            1
12                             No            1
13                             No            1
14                             No            0
```

2 Preprocessing

First, some inspections and adjustments must be made to the dataset to ensure smooth sailing when proceeding to the analysis.

[2]:
```
# Feature type and checking for missing values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45000 entries, 0 to 44999
Data columns (total 14 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   person_age                      45000 non-null  float64
 1   person_gender                   45000 non-null  object
 2   person_education                45000 non-null  object
 3   person_income                   45000 non-null  float64
 4   person_emp_exp                  45000 non-null  int64
 5   person_home_ownership           45000 non-null  object
 6   loan_amnt                       45000 non-null  float64
 7   loan_intent                     45000 non-null  object
 8   loan_int_rate                   45000 non-null  float64
 9   loan_percent_income             45000 non-null  float64
 10  cb_person_cred_hist_length      45000 non-null  float64
 11  credit_score                    45000 non-null  int64
 12  previous_loan_defaults_on_file  45000 non-null  object
 13  loan_status                     45000 non-null  int64
dtypes: float64(6), int64(3), object(5)
memory usage: 4.8+ MB
```

[3]:
```python
# Summary of numerical features
df.describe().T
```

[3]:

	count	mean	std	min	\
person_age	45000.0	27.764178	6.045108	20.00	
person_income	45000.0	80319.053222	80422.498632	8000.00	
person_emp_exp	45000.0	5.410333	6.063532	0.00	
loan_amnt	45000.0	9583.157556	6314.886691	500.00	
loan_int_rate	45000.0	11.006606	2.978808	5.42	
loan_percent_income	45000.0	0.139725	0.087212	0.00	
cb_person_cred_hist_length	45000.0	5.867489	3.879702	2.00	
credit_score	45000.0	632.608756	50.435865	390.00	
loan_status	45000.0	0.222222	0.415744	0.00	

	25%	50%	75%	max
person_age	24.00	26.00	30.00	144.00
person_income	47204.00	67048.00	95789.25	7200766.00
person_emp_exp	1.00	4.00	8.00	125.00
loan_amnt	5000.00	8000.00	12237.25	35000.00
loan_int_rate	8.59	11.01	12.99	20.00
loan_percent_income	0.07	0.12	0.19	0.66
cb_person_cred_hist_length	3.00	4.00	8.00	30.00
credit_score	601.00	640.00	670.00	850.00
loan_status	0.00	0.00	0.00	1.00

- There are some outliers in age, income, and employment experience. For age and employment experience, this is possibly due to the applicant never being removed from the system or from a data entry problem. An annual income of $7,200,766, while certainly an outlier, does not feel erroneous. The loan was most likely taken out for financial or personal investments. We will inspect these outliers later.
- Nonetheless, given the quartile values, these outliers do not appear to have high leverage.
- Quartiles in age reveal that the dataset is skewed towards younger borrowers.
- Loan status is a 0 or 1 response. Therefore, a mean of 0.2222 implies that 22.22% of applicants successfully repaid the loan and 77.78% of them defaulted.

[4]:
```python
# Sort columns by missing percentage from highest to lowest
no_entry = df.isna().mean().sort_values(ascending = False)
no_entry[no_entry > 0]
```

[4]:
```
Series([], dtype: float64)
```

[5]:
```python
# Checking for unique values
print("Duplicated row count:", df.duplicated().sum())
```

```
Duplicated row count: 0
```

This dataset has no missing entries in each feature and each entry is unique.

[6]:
```python
# Number of unique values in each feature
num_unique = df.nunique().sort_values(ascending = False)
num_unique

# Identify features with at most 9 classes
for col in num_unique.index:
    uniq_count = num_unique[col]
    if uniq_count < 10:
        uniq_vals = df[col].dropna().unique()
        print(f"{col}: {uniq_count} ({list(uniq_vals)})")
    else:
        print(f"{col}: {uniq_count}")
```

```
person_income: 33989
loan_amnt: 4483
loan_int_rate: 1302
credit_score: 340
loan_percent_income: 64
person_emp_exp: 63
person_age: 60
cb_person_cred_hist_length: 29
loan_intent: 6 (['PERSONAL', 'EDUCATION', 'MEDICAL', 'VENTURE',
'HOMEIMPROVEMENT', 'DEBTCONSOLIDATION'])
person_education: 5 (['Master', 'High School', 'Bachelor', 'Associate',
'Doctorate'])
person_home_ownership: 4 (['RENT', 'OWN', 'MORTGAGE', 'OTHER'])
person_gender: 2 (['female', 'male'])
previous_loan_defaults_on_file: 2 (['No', 'Yes'])
loan_status: 2 ([1, 0])
```

Each categorical variable has no more than 6 classes. Moreover, education is an ordinal variable, so we will reorder it to parity student progression and for easier visualization/interpretation later. Additionally, we will need to separate categorical from quantiative features for encoding later.

[7]:
```python
# Reorder education to match typical progression
ord_edu = ['High School', 'Associate', 'Bachelor', 'Master', 'Doctorate']
df['person_education'] = df['person_education'].astype(pd.
 ↪CategoricalDtype(categories = ord_edu, ordered = True))

# Build quantiative and qualitative exclusive dataframes
cat_cols = df.select_dtypes(include = ["object"]).columns.tolist()
num_cols = df.select_dtypes(include = [np.number]).columns.tolist()
cat_cols, num_cols
```

[7]:
```
(['person_gender',
  'person_home_ownership',
  'loan_intent',
  'previous_loan_defaults_on_file'],
```

```
 ['person_age',
  'person_income',
  'person_emp_exp',
  'loan_amnt',
  'loan_int_rate',
  'loan_percent_income',
  'cb_person_cred_hist_length',
  'credit_score',
  'loan_status'])
```

Lastly, we will look at the outliers mentioned earlier.

[8]:
```python
# Identifying borrowers with outliers in age
age_outlier = df[df['person_age'] > 100]
age_outlier
```

[8]:
```
       person_age person_gender person_education  person_income  \
81          144.0          male         Bachelor       300616.0
183         144.0          male        Associate       241424.0
575         123.0        female      High School        97140.0
747         123.0          male         Bachelor        94723.0
32297       144.0        female        Associate      7200766.0
37930       116.0          male         Bachelor      5545545.0
38113       109.0          male      High School      5556399.0

       person_emp_exp person_home_ownership  loan_amnt loan_intent  \
81                125                  RENT     4800.0     VENTURE
183               121              MORTGAGE     6000.0   EDUCATION
575               101                  RENT    20400.0   EDUCATION
747               100                  RENT    20000.0     VENTURE
32297             124              MORTGAGE     5000.0    PERSONAL
37930              93              MORTGAGE     3823.0     VENTURE
38113              85              MORTGAGE     6195.0     VENTURE

       loan_int_rate  loan_percent_income  cb_person_cred_hist_length  \
81             13.57                 0.02                         3.0
183            11.86                 0.02                         2.0
575            10.25                 0.21                         3.0
747            11.01                 0.21                         4.0
32297          12.73                 0.00                        25.0
37930          12.15                 0.00                        24.0
38113          12.58                 0.00                        22.0

       credit_score previous_loan_defaults_on_file  loan_status
81              789                            No            0
183             807                            No            0
575             805                           Yes            0
747             714                           Yes            0
```

```
32297          850                            No            0
37930          708                            No            0
38113          792                            No            0
```

[9]:
```python
# Identifying borrowers wtih outliers in income
income_outlier = df[df['person_income'] > 2000000]
income_outlier
```

[9]:
```
       person_age person_gender person_education  person_income  \
30049        42.0          male      High School      2448661.0
32297       144.0        female        Associate      7200766.0
32497        63.0          male      High School      2139143.0
32546        60.0        female      High School      2280980.0
37175        42.0          male      High School      2012954.0
37930       116.0          male         Bachelor      5545545.0
38113       109.0          male      High School      5556399.0

       person_emp_exp person_home_ownership  loan_amnt loan_intent  \
30049              16                   RENT     8450.0     VENTURE
32297             124              MORTGAGE     5000.0    PERSONAL
32497              43                   RENT    12025.0   EDUCATION
32546              37              MORTGAGE     1500.0    PERSONAL
37175              19              MORTGAGE     9121.0     VENTURE
37930              93              MORTGAGE     3823.0     VENTURE
38113              85              MORTGAGE     6195.0     VENTURE

       loan_int_rate  loan_percent_income  cb_person_cred_hist_length  \
30049          12.29                 0.00                        15.0
32297          12.73                 0.00                        25.0
32497          14.27                 0.01                        30.0
32546          11.01                 0.00                        21.0
37175          11.74                 0.00                        16.0
37930          12.15                 0.00                        24.0
38113          12.58                 0.00                        22.0

       credit_score previous_loan_defaults_on_file  loan_status
30049           635                           Yes            0
32297           850                            No            0
32497           724                            No            0
32546           682                           Yes            0
37175           664                            No            0
37930           708                            No            0
38113           792                            No            0
```

Some outliers in age are also outliers in income. It is also worth noting that 6 of the outliers in income are due to venture and personal interests, and that 5 of them did not go beyond a high school diploma.

[10]:
```python
df['loan_status'].value_counts(dropna = False)
```

[10]:
```
loan_status
0    35000
1    10000
Name: count, dtype: int64
```

The dataframe summary already revealed this; a class imbalance does exist between paid-on-time and default borrowers (with a majority in the default class). While not extreme, we should still be mindful when analyzing classification results.

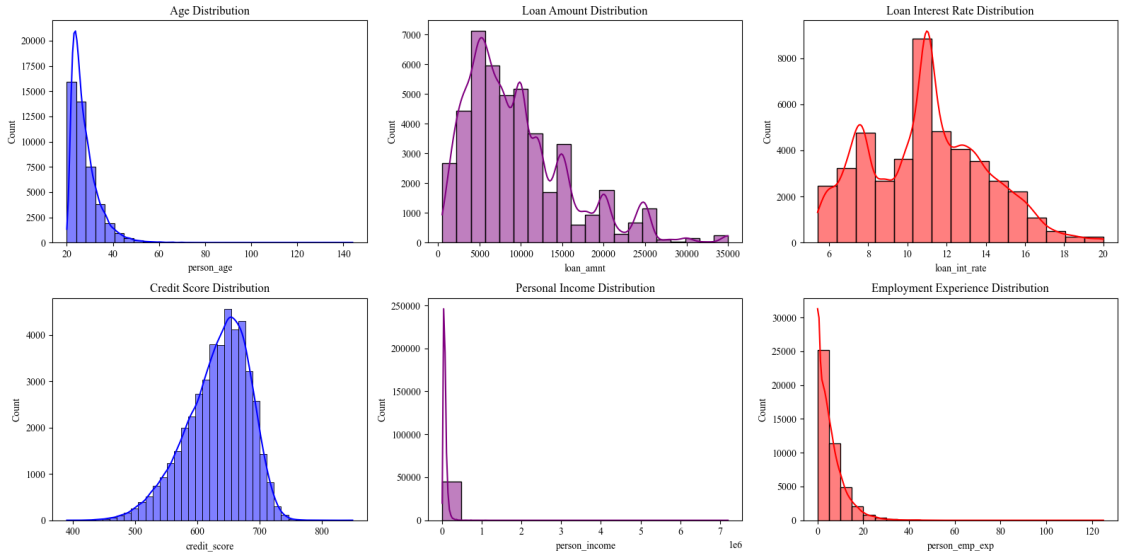
3 Exploratory Data Analysis

We are primarily interested in three sets of analyses:

- Count by quantitative features
- Count of loan status by categorical features
- Interactions between features (quantitative & categorical)

[11]:
```
fig, axes = plt.subplots(2, 3, figsize=(16, 8))

# Age Density distribution
age_dens = sns.histplot(data = df, bins = 30, x = 'person_age', kde = True, color =
 ↪'blue', ax=axes[0, 0])
axes[0, 0].set_title("Age Distribution")

# Loan Amount distogram
loan_dens = sns.histplot(data = df, bins = 20, x = 'loan_amnt', kde = True, color =
 ↪'purple', ax=axes[0, 1])
axes[0, 1].set_title("Loan Amount Distribution")

# Loan interest rate distribution
ir_dens = sns.histplot(data = df, bins = 15, x = 'loan_int_rate', kde = True, color
 ↪= 'red', ax=axes[0, 2])
axes[0, 2].set_title("Loan Interest Rate Distribution")

# Credit Score distribution
cs_dens = sns.histplot(data = df, bins = 40, x = 'credit_score', kde = True, color =
 ↪'blue', ax=axes[1, 0])
axes[1, 0].set_title("Credit Score Distribution")

# Personal income distribution
pi_dens = sns.histplot(data = df, bins = 15, x = 'person_income', kde = True, color
 ↪= 'purple', ax=axes[1, 1])
axes[1, 1].set_title("Personal Income Distribution")

# Employment experience distribution
emp_dens = sns.histplot(data = df, bins = 25, x = 'person_emp_exp', kde = True,
 ↪color = 'red', ax=axes[1, 2])
```

```
axes[1, 2].set_title("Employment Experience Distribution")

plt.tight_layout()
```

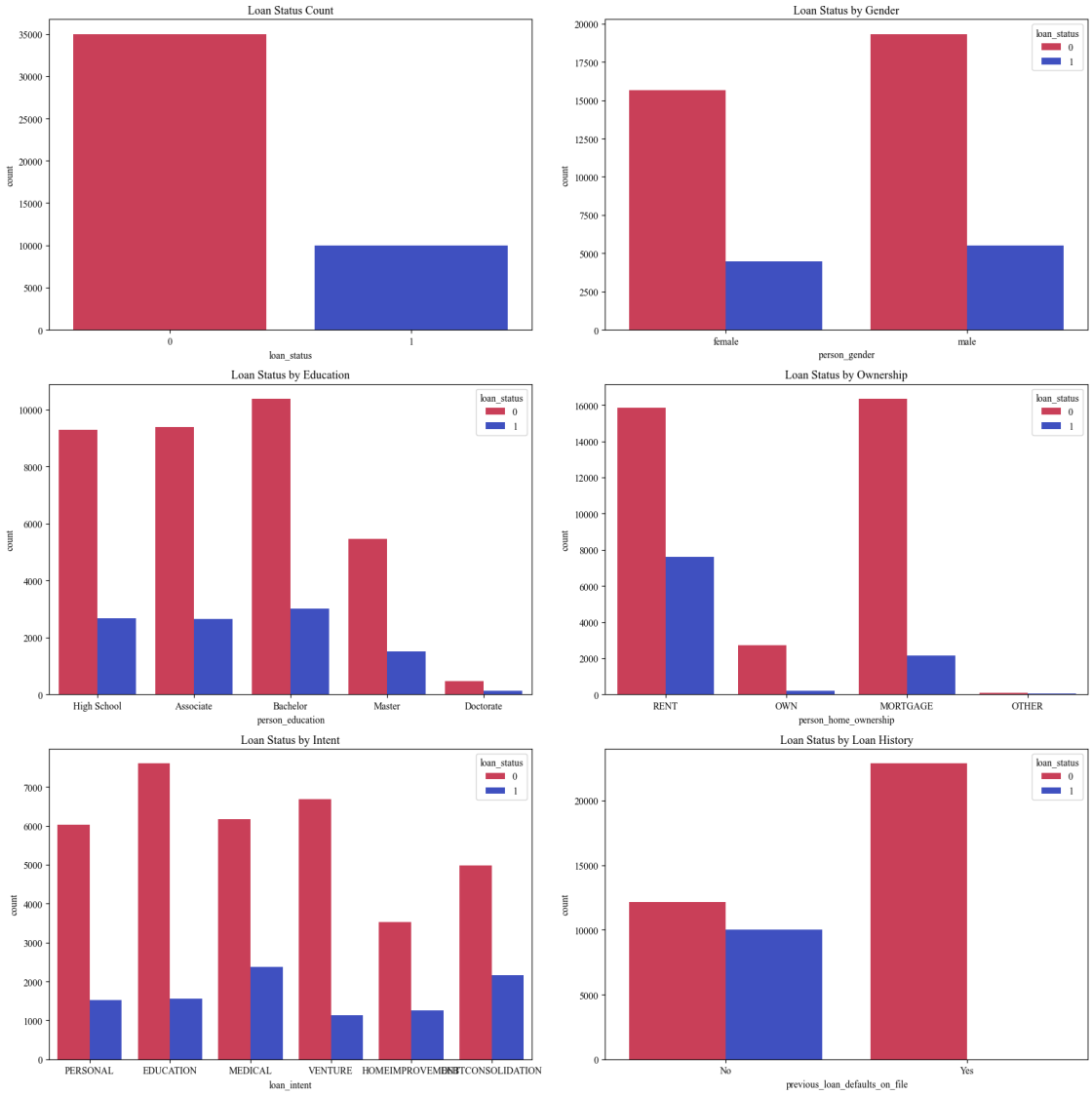


- Much like the dataframe summary implied, age is skewed towards younger borrowers. There is a strict decline in borrower count in each subsequent age bin.
- Most loans are requested in smaller amounts (0-20,000 USD).
- Interest rates are mostly consistent, with a peak at the ~11% bin. From there, larger interest rates become increasingly rare (to be expected).
- Credit scores almost resemble a normal distribution, with the mean at 650. However, there is a slight skew towards larger credit scores.
- Almost every borrower in the dataset earns less than 500,000 USD annually.
- Employment experience follows a strict decline, very close in shape to age count. Intuitively this makes sense: older borrowers will have more years of work experience. They likely have positive correlation of a decent magnitude.

[12]:
```
fig, axes = plt.subplots(3, 2, figsize=(16, 16))

# Loan Status count
sns.countplot(data = df, x = 'loan_status', ax=axes[0, 0], palette = ('#E0284A',
 ↪'#2940D6'))
axes[0, 0].set_title("Loan Status Count")

# Loan Status by gender
sns.countplot(data = df, x = 'person_gender', hue = 'loan_status', ax=axes[0, 1],
 ↪palette = ('#E0284A', '#2940D6'))
axes[0, 1].set_title("Loan Status by Gender")

# Loan Status by education
sns.countplot(data = df, x = 'person_education', hue = 'loan_status', ax=axes[1,
 ↪0], palette = ('#E0284A', '#2940D6'))
```

```
axes[1, 0].set_title("Loan Status by Education")

# Loan Status by ownership
sns.countplot(data = df, x = 'person_home_ownership', hue = 'loan_status', ax =
 ↪axes[1, 1], palette = ('#E0284A', '#2940D6'))
axes[1, 1].set_title("Loan Status by Ownership")

# Loan Status by intent
sns.countplot(data = df, x = 'loan_intent', hue = 'loan_status', ax = axes[2, 0],
 ↪palette = ('#E0284A', '#2940D6'))
axes[2, 0].set_title("Loan Status by Intent")

# Loan Status by loan history
sns.countplot(data = df, x = 'previous_loan_defaults_on_file', hue = 'loan_status',
 ↪ax = axes[2, 1], palette = ('#E0284A', '#2940D6'))
axes[2, 1].set_title("Loan Status by Loan History")

plt.tight_layout()
```

(Output on following page)



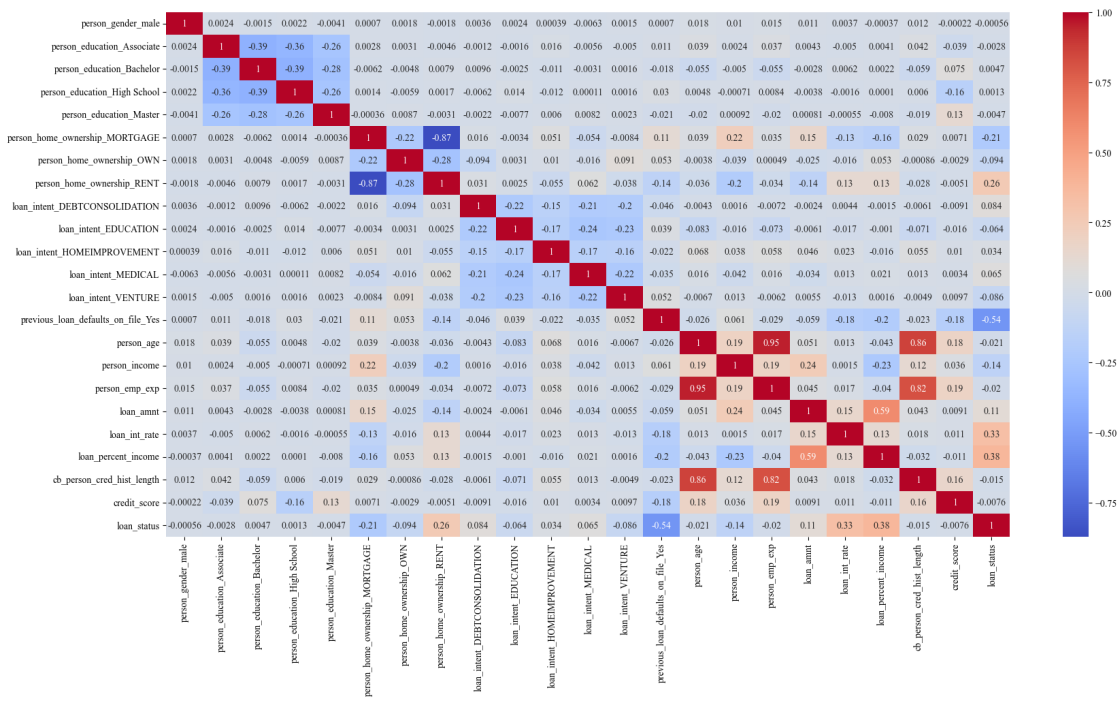
- There is fair representation between males and female borrowers and they share a roughly equal distribution of repaid and defaulted loans.
- Within each education class, the proportion of defaulted over paid is pretty similar.
- Almost all borrowers are renting, own, or mortgaging home property. Borrowers who own their homes appear least likely to default, while renters show higher default proportions. Mortgage holders lie between the two.
- Loans taken for debt consolidation appear to exhibit higher default proportions relative to other loan purposes.
- Most importantly, borrowers with previously loan defaults have substantially higher default rates than those without previous defaults. Therefore, we would predict all future borrowers to default if they have past defaults on file, making it the strongest predictor in this dataset and likely to dominate all predictive models. While this makes sense, we also must proceed with caution with these results, namely avoiding potential bias.

Lastly, to decide feature interactions, a correlation matrix will be constructed after encoding the categorical covariates.

[13]:
```
# One-hot encoding
encode_cols = ['person_gender', 'person_education', 'person_home_ownership',
 ↪'loan_intent', 'previous_loan_defaults_on_file']
encoder = OneHotEncoder(drop = ['female', 'Doctorate', 'OTHER', 'PERSONAL', 'No'],
 ↪sparse_output = False)
encoded_data = encoder.fit_transform(df[encode_cols])
col_names = encoder.get_feature_names_out(encode_cols)
df_encoded = pd.concat([
    pd.DataFrame(encoded_data, columns = col_names),
    df[num_cols],
], axis = 1)
df_encoded
```

[14]:
```
plt.figure(figsize = (20, 10))
sns.heatmap(df_encoded.corr(), annot = True, cmap = 'coolwarm')
```

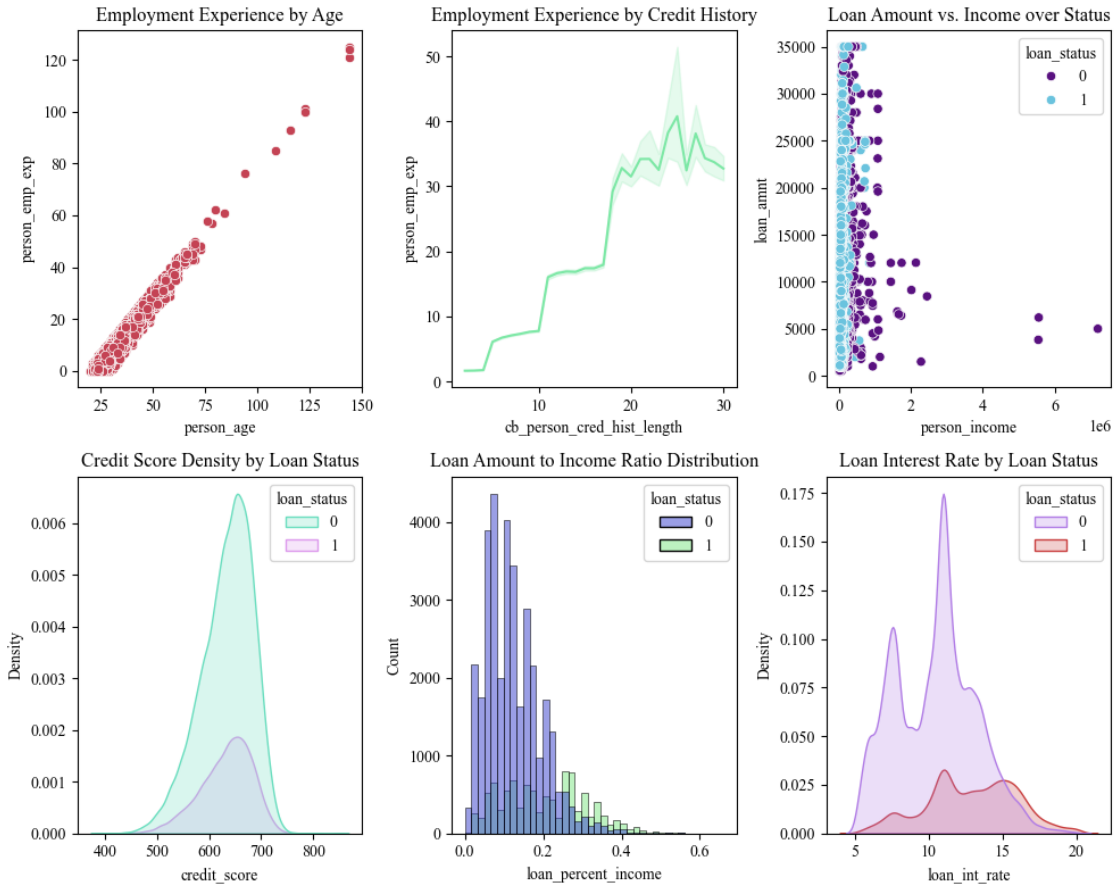
[14]: <Axes: >



We are primarily concerned with the correlations on the bottom right quadrant. The correlations for education, ownership, and intent are all negative by construction. The following code and charts will reflect said interest.

[15]:
```
fig, axes = plt.subplots(2, 3, figsize = (10, 8))

# Employment experience by age
sns.scatterplot(data = df, x = 'person_age', y = 'person_emp_exp', color =
 ↪'#C44354', ax = axes[0, 0])
axes[0, 0].set_title("Employment Experience by Age")
```

```
# Employment experience by credit history
sns.lineplot(data = df, x = 'cb_person_cred_hist_length', y = 'person_emp_exp',
 ↪color = '#7DE8A6', ax = axes[0, 1])
axes[0, 1].set_title("Employment Experience by Credit History")

# Loan amount vs. income over loan status
sns.scatterplot(data = df, x = 'person_income', y = 'loan_amnt', hue =
 ↪'loan_status', palette = ('#591180', '#6AC3DE'), ax = axes[0, 2])
axes[0, 2].set_title("Loan Amount vs. Income over Status")

# Credit score density by loan status
sns.kdeplot(data = df, x = 'credit_score', hue = 'loan_status', fill = True,
 ↪palette = ('#6ADEBF', '#DD9AED'), ax = axes[1, 0])
axes[1, 0].set_title('Credit Score Density by Loan Status')

# Loan to income ratio count
sns.histplot(data = df, x = 'loan_percent_income', bins = 40, kde = False, hue =
 ↪'loan_status', palette = ('#373FCC', '#7CE683'), ax = axes[1, 1])
axes[1, 1].set_title('Loan Amount to Income Ratio Distribution')

# Loan interest rate by status
sns.kdeplot(data = df, x = 'loan_int_rate', hue = 'loan_status', fill = True,
 ↪palette = ('#B17CE6', '#C93E3E'), ax = axes[1, 2])
axes[1, 2].set_title('Loan Interest Rate by Loan Status')

plt.tight_layout()
```



- With $\rho = 0.95$, employment experience and age appear to have a positive, linear relationship. Thus, there is potential for overfitting training data in models which include both features. As a final measure, we will check their VIFs relative to each other.
- Employment experience is also positively correlated with credit history ($\rho = 0.82$), but smaller in magnitude compared to age. That being said, age is, by transitivity, also positively correlated with credit history in some capacity ($\rho = 0.86$).
- Loan amount vs annual income, controlling over loan status ($\rho = 0.59$). We see that a lot of borrowers with greater annual income are more likely to default and they requested loans of smaller amounts. That said, 77.78% of the borrowers have defaulted, leaving less chance for variation among those who repaid. When looking at the loan amount to income ratio, it is worth noting that borrowers with a higher loan amount to income ratio appear to be more likely to repay their loan. Once again, this is possibly due to the class imbalance we saw earlier.
- Credit score, when adjusting for loan status, surprisingly have a similar "bell curve" shape between both default and repaid applicants, although borrowers who repaid are more skewed toward higher credit scores. For this reason, credit score is projected to have some influence on loan statuses.
- Loan status and interest rate have a positive correlation, but lower in magnitude with respect to the other interactions we have mentioned ($\rho = 0.33$). The spread is less extreme for borrowers who repaid compared to ones who defaulted. Interest rate can have influence on predicting loan status with its positive correlation (harder to repay loans with higher interest rates), but it is difficult to do so without knowing the term of the loan.

[16]:
```python
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

lin_cols = ['person_age', 'person_emp_exp', 'cb_person_cred_hist_length']
X = df[lin_cols]
X = sm.add_constant(X)
vif_df = pd.DataFrame()
vif_df['features'] = X.columns
vif_df['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif_df.sort_values(by = 'VIF', ascending = False))
```

```
                    features         VIF
0                      const  181.025244
1                 person_age   14.002649
2              person_emp_exp   11.224897
3  cb_person_cred_hist_length    3.891740
```

Age and employment experience, as expected, have high VIFs (>10), confirming their multicollinearity. In the next section, we will construct a model without these features and see how it compares.

4 Models & Methodology

Now that we have a nuanced understanding of the dataset, we can begin implementing predictive models and determine if they validate any preliminary analyses made so far. Because loan status is a 0/1 response variable, we are restricted to binary classifiers. More precisely, the models to be constructed are:

1. **Logistic Regression** (3 candidates: All features, without age/employment experience, and lasso). Logistic Regression is always a great starting ground for binary classification with easy interpretability and variability. They can also readily compute probabilities of class assignment, which would make the upcoming sections more efficient.

2. **Linear Discriminant Analysis (LDA)**: As a dimensionality reduction method, LDA can be useful with complex datasets. This dataset has 13 features; by lowering the dimension, the computational cost is lowered and has potential for reliably separating borrowers who repaid on time against those who defaulted.

3. **Ensemble Methods** (2 candidates: Random forest and boosting): A standard random forest is easy to implement and generally fixes the overfitting dilemma individual decision trees typically face. Boosting, as an iterative method, will understand and correct from errors made in previous trees. Therefore, it is an extremely powerful model for dealing with class imbalances.

Because the dataset is not incredibly large, we are not concerned with the computational time required to receive results.

Each model will be assessed under accuracy, F1-score, and AUC measurements, with the latter two holding more weight. A higher F1-score implies the model lacks type I/II errors and is well accounting for the minority class. The same applies to AUC, where we want better separability capacity between the two target classes.

We are also concerned with the presence of type I errors on the test data. Type I errors reflect false positives when classifying test data; in our case, a type I error suggests that a borrower who would have

defaulted is actually classified as "paid-on-time"; therefore, loans are issued to high-risk borrowers and will result in serious financial losses. This is an incredibly dangerous financial circumstance; lenders do not want to approve loans to borrowers who have a noticeable chance of defaulting. By contrast, type II errors correspond to rejecting borrowers who would have repaid their loans, resulting in a missed lending opportunity; however, it does not directly incur losses to the lender. Consequently, minimizing type I errors in credit risk modeling are typically more important.

Additionally, cross-validation and bootstrapping confidence intervals will verify that the diagonistics apply to random samples of the dataset.

After reviewing the models on diagonistics of predictive power, the one with highest performance will be used in credit risk assessment modeling.

4.1 Logistic Regression

As stated previously, we are implementing three logistic regression models: - A model with all 13 features - A model excluding age and employment experience - A lasso (L1) regularized model

If the results from EDA hold, we should expect a model without age and employment experience to perform better than one with every feature. The lasso model should lie between them or surpass the modified model.

For all models, we will use a 75% training data and 25% test data split.

4.1.1 Base Model (All Features)

[17]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn import metrics

# Splitting the data, including all features
X = df_encoded.drop(['loan_status'], axis = 1)
y = df_encoded['loan_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
 ↪random_state = 0)

log_model = LogisticRegression(random_state = 0)
log_model.fit(X_train, y_train)
y_predlog = log_model.predict(X_test)

print(f"Logistic Regression Model Accuracy = {metrics.accuracy_score(y_test,
 ↪y_predlog):.2%}")
```

```
Logistic Regression Model Accuracy = 84.60%
```

4.1.2 Modified Model (Excluding Age & Employment Experience)

[18]:
```
# Logistic Regression excluding age and employment experience
X2 = df_encoded.drop(['person_age', 'person_emp_exp', 'loan_status'], axis = 1)
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y, test_size = 0.25,
 ↪random_state = 0)
```

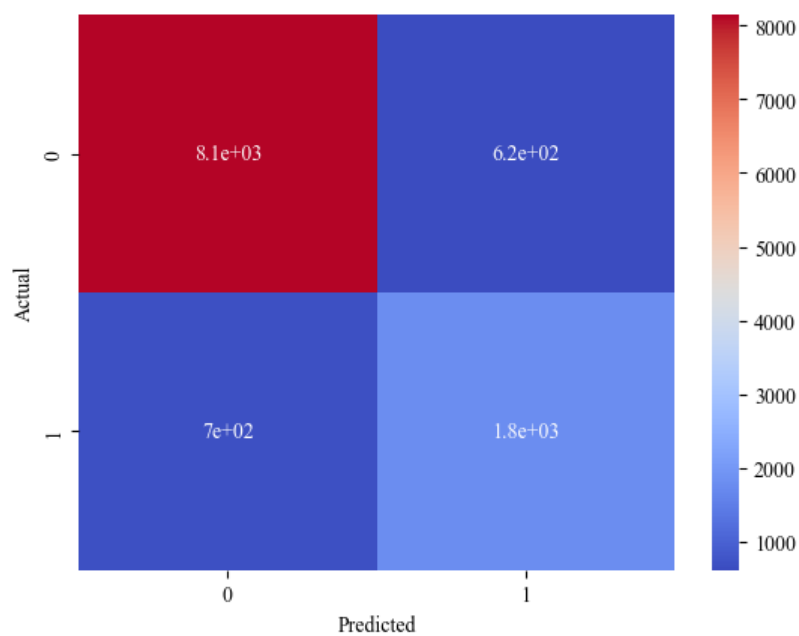
```
log_model2 = LogisticRegression(random_state = 0)
log_model2.fit(X2_train, y2_train)
y2_predlog = log_model2.predict(X2_test)

print(f"Modified Logisitc Regression Model Accuracy = {metrics.
↪accuracy_score(y2_test, y2_predlog):.2%}")
```

```
Modified Logisitc Regression Model Accuracy = 88.24%
```

[19]:
```
# Confusion matrix
print(metrics.confusion_matrix(y_test, y2_predlog))
```

```
[[8143  620]
 [ 703 1784]]
```

[20]:
```
sns.heatmap(metrics.confusion_matrix(y2_test, y2_predlog), annot = True, cmap =
↪'coolwarm')
plt.xlabel('Predicted')
plt.ylabel('Actual');
```



[21]:
```
# Classification
print(metrics.classification_report(y2_test, y2_predlog, target_names = ['Default',
↪'Paid-on-time']))
```

```
              precision    recall  f1-score   support

     Default       0.92      0.93      0.92      8763
```

```
Paid-on-time       0.74      0.72      0.73      2487

    accuracy                           0.88     11250
   macro avg       0.83      0.82      0.83     11250
weighted avg       0.88      0.88      0.88     11250
```

While the accuracy of 88% is promising after dropping age and employment experience, it must be interpreted with caution due to the strong class imbalance. As a result, a model can obtain high accuracy by favoring/prioritizing the majority class. Although the model performs well in classifying defaults (precision = 92%, recall = 93%), its performance falls for borrowers who paid-on-time. The F1-score for the paid-on-time class is 0.73, reflecting a moderate predictive ability and indicating that a large portion of repaid loans are misclassified. More precisely, the recall of 0.72 suggests that 28% of borrowers who repay their loans are incorrectly predicted to default, leaving room for improvement.

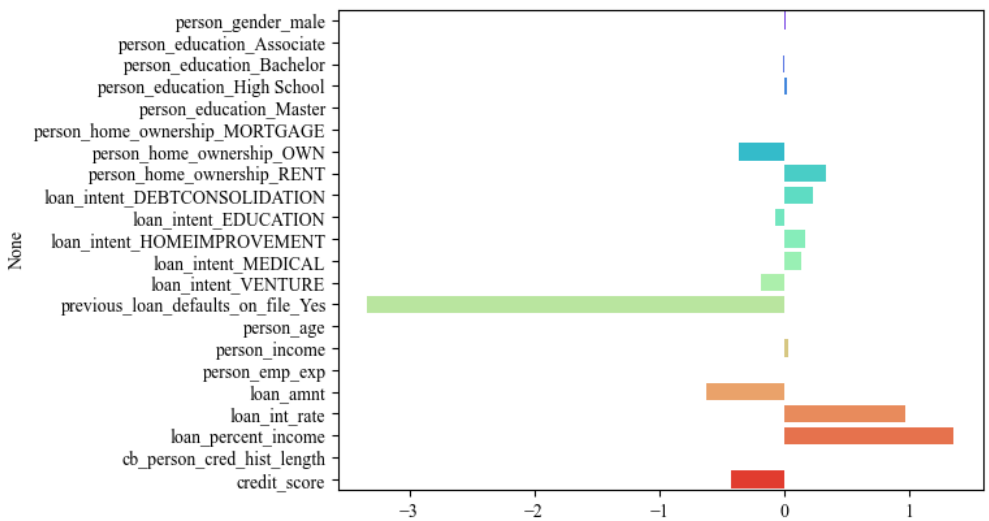
4.1.3 L1-Regularization Model

[22]:
```python
# L1 Regularization: Fitting
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegressionCV

X_scaled = StandardScaler().fit_transform(X)
Xs_train, Xs_test, ys_train, ys_test = train_test_split(X_scaled, y, test_size = 0.
↪25, random_state = 0)

# Optimizing the regularization coefficient
lasso_cv = LogisticRegressionCV(
    penalty = 'l1',
    solver = 'liblinear',
    cv = 5,
    scoring = 'roc_auc',
    Cs = 10,
    max_iter = 10
)

lasso_cv.fit(Xs_train, y_train)
print("Optimal C:", lasso_cv.C_)

L1log = LogisticRegression(penalty = 'l1', n_jobs = -1, solver = 'liblinear', C = 0.
↪04641589)
L1log.fit(Xs_train, ys_train)

# Printing Coefficients
print(L1log.coef_)
print(L1log.intercept_)

y_predlogL1 = L1log.predict(Xs_test)
```

```
print(f"Lasso Logistic Regression Model Accuracy = {metrics.accuracy_score(y_test,
↪y_predlogL1):.2%}")

sns.barplot(x = L1log.coef_[0], y = X_train.columns, palette = 'rainbow')
```

```
Optimal C: [166.81005372]
[[ 1.30940451e-02  0.00000000e+00 -3.90661611e-03  2.53858258e-02
   1.19407007e-03  0.00000000e+00 -3.66210759e-01  3.30393452e-01
   2.34527329e-01 -7.23839028e-02  1.67691542e-01  1.38305002e-01
  -1.82765142e-01 -3.33510989e+00  0.00000000e+00  3.38965894e-02
   0.00000000e+00 -6.20855575e-01  9.71332544e-01  1.35695588e+00
   0.00000000e+00 -4.23350910e-01]]
[-4.10601809]
Lasso Logistic Regression Model Accuracy = 89.36%
```

[22]: <Axes: ylabel='None'>

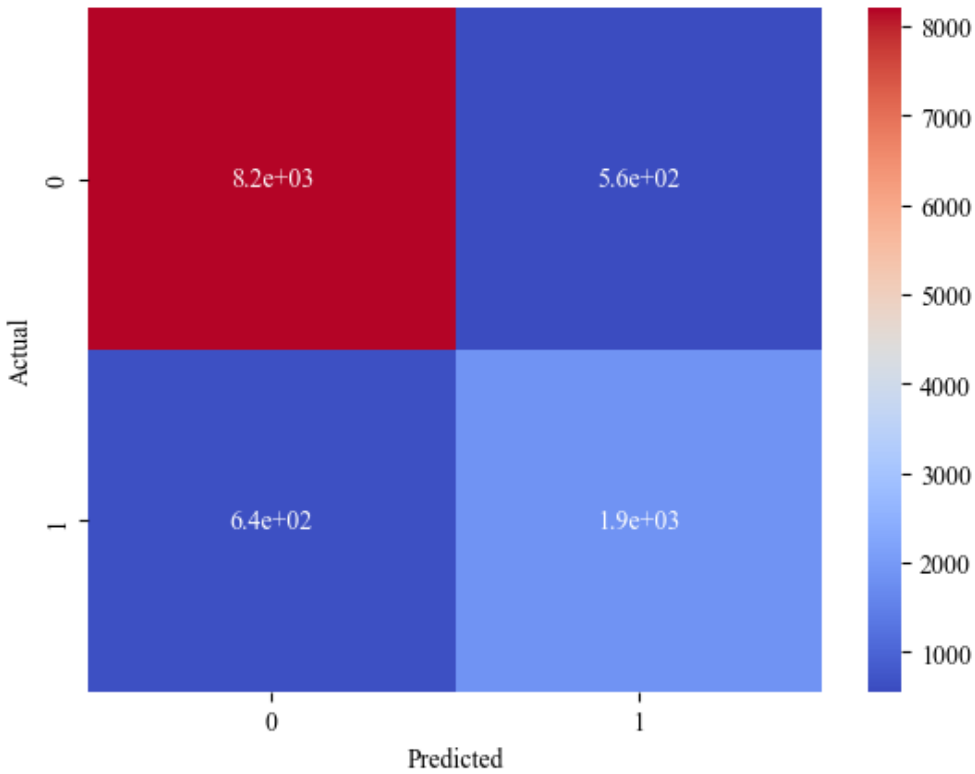


As expected, previous defaults constitutes the most critical feature in predicting defaults. Other features such as loan amount, %income, interest rate, and credit score are also important features (as revealed through EDA). The L1-regularized model decided to drop education classes, as well as age, employment experience, and credit history length. These are features that were credited with a noticeable VIF and were therefore removed to avoid multicollinearity.

[23]:
```
# Confusion matrix
print(metrics.confusion_matrix(y_test, y_predlogL1))
```

```
[[8202  561]
 [ 636 1851]]
```

[24]:
```
sns.heatmap(metrics.confusion_matrix(y2_test, y_predlogL1), annot = True, cmap =
↪'coolwarm')
plt.xlabel('Predicted')
plt.ylabel('Actual');
```



[25]:
```
# Classification
print(metrics.classification_report(y_test, y_predlogL1, target_names = ['Default',
↪'Paid-on-time']))
```

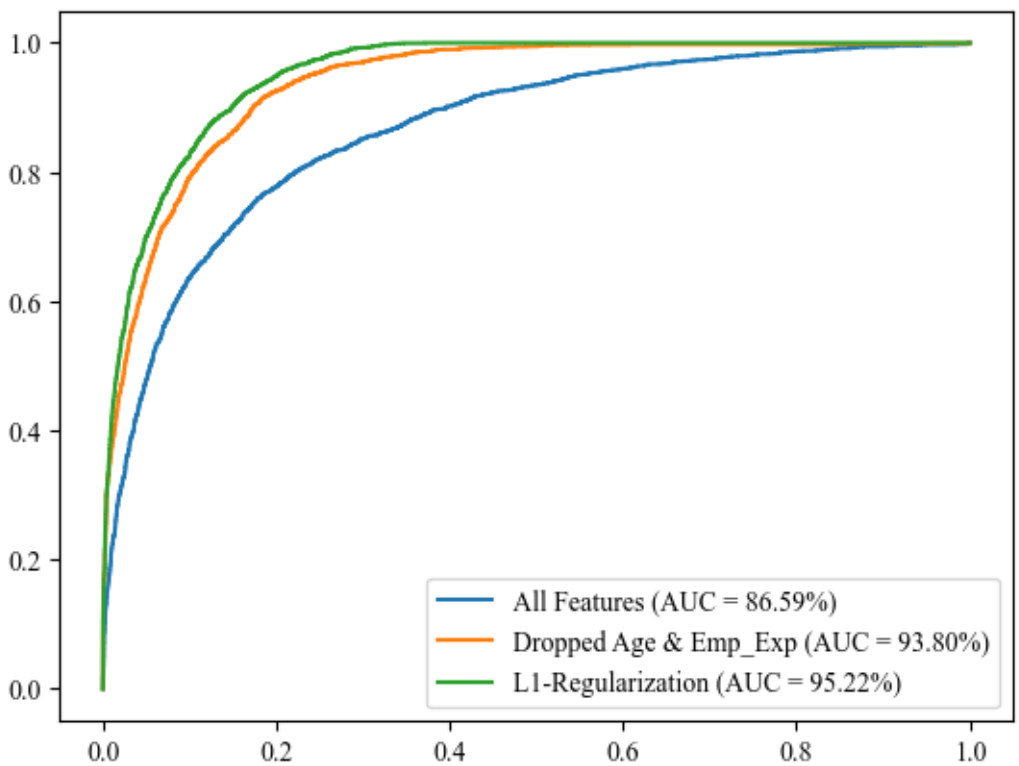
```
              precision    recall  f1-score   support

     Default       0.93      0.94      0.93      8763
Paid-on-time       0.77      0.74      0.76      2487

    accuracy                           0.89     11250
   macro avg       0.85      0.84      0.84     11250
weighted avg       0.89      0.89      0.89     11250
```

Compared to just removing age and employment experience, lasso regularization is a slight improvement in F1-score for both target classes.

[26]:
```
# ROC Curves
y_problog = log_model.predict_proba(X_test)[:, 1]
fpr, tpr, _ = metrics.roc_curve(y_test, y_problog)
auc = metrics.roc_auc_score(y_test, y_problog)

y_problog2 = log_model2.predict_proba(X2_test)[:, 1]
fpr2, tpr2, _ = metrics.roc_curve(y2_test, y_problog2)
auc2 = metrics.roc_auc_score(y2_test, y_problog2)
```

```
y_problog3 = L1log.predict_proba(Xs_test)[:, 1]
fpr3, tpr3, _ = metrics.roc_curve(ys_test, y_problog3)
auc3 = metrics.roc_auc_score(ys_test, y_problog3)

plt.plot(fpr, tpr, label = f"All Features (AUC = {auc:.2%})")
plt.plot(fpr2, tpr2, label = f"Dropped Age & Emp_Exp (AUC = {auc2:.2%})")
plt.plot(fpr3, tpr3, label = f"L1-Regularization (AUC = {auc3:.2%})")

plt.legend()
```

[26]: <matplotlib.legend.Legend at 0x1d9dc6869c0>



As depicted by their AUCs, L1-regularization exhibits the best degree of separability, with "dropped age & emp_exp" not too far behind. That being said, L1-regularization has the highest chance of distinguishing between the default class and paid-on-time class (95.22%).

Next, we will verify the accuracy and AUC results through 10-fold cross validation and 95% confidence interval bootstrapping.

[27]:
```
# 10-fold cross validation
from sklearn.model_selection import cross_val_score
```

```
scores_reg = cross_val_score(log_model, X_train, y_train, cv = 10, scoring =
 ↪'accuracy')
scores_dropped = cross_val_score(log_model2, X2_train, y2_train, cv = 10, scoring =
 ↪'accuracy')
scores_L1 = cross_val_score(L1log, Xs_train, ys_train, cv = 10, scoring = 'accuracy')

print(f"Cross-validation scores without changes: {scores_reg}")
print(f"Mean accuracy without changes: {np.mean(scores_reg):.4f}")

print(f"Cross-validation scores after dropping : {scores_dropped}")
print(f"Mean accuracy after dropping: {np.mean(scores_dropped):.4f}")

print(f"Cross-validation scores with L1-regularization: {scores_L1}")
print(f"Mean accuracy with L1-regularization: {np.mean(scores_L1):.4f}")
```

```
Cross-validation scores without changes: [0.84266667 0.83022222 0.8397037
0.82874074 0.83081481 0.85155556
 0.85007407 0.82637037 0.82281481 0.82607407]
Mean accuracy without changes: 0.8349
Cross-validation scores after dropping : [0.88533333 0.87674074 0.87259259
0.88237037 0.88651852 0.86607407
 0.87111111 0.87288889 0.87733333 0.87703704]
Mean accuracy after dropping: 0.8768
Cross-validation scores with L1-regularization: [0.90103704 0.89985185
0.89777778 0.89007407 0.90192593 0.89392593
 0.89807407 0.89688889 0.89362963 0.89511111]
Mean accuracy with L1-regularization: 0.8968
```

[28]:
```
# Bootstrapping over 1000 samples
from sklearn.metrics import roc_auc_score
def bootstrap_auc(y_true, y_pred_proba, n_boot = 1000, random_state = 42):
    """ Returns the original AUC, mean AUC, and the lower and upper
        limits of the 95% confidence interval for the AUC
        Arguments: actual value of test data, predicted value of test
        data
        Output: original and mean AUC, as well as the bounds of the
        95% confidence interval
    """
    rng = np.random.default_rng(random_state)
    n = len(y_true)
    aucs = []
    for _ in range(n_boot):
        idx = rng.integers(0, n, n)
        if len(np.unique(y_true.iloc[idx])) < 2:
            continue

        auc = roc_auc_score(y_true.iloc[idx], y_pred_proba[idx])
```

```
        aucs.append(auc)
    aucs = np.array(aucs)
    lowerperc = np.percentile(aucs, 2.5)
    upperperc = np.percentile(aucs, 97.5)
    return {
        "original_auc": roc_auc_score(y_true, y_pred_proba),
        "mean_auc": aucs.mean(),
        "ci_lower": lowerperc,
        "ci_upper": upperperc,
    }

results = bootstrap_auc(y2_test, y_problog2)
print("Mean Bootstrap AUC:", results["mean_auc"])
print("95% CI:", results["ci_lower"], "-", results["ci_upper"])

results_L1 = bootstrap_auc(ys_test, y_problog3)
print("Mean Bootstrap L1 AUC:", results_L1["mean_auc"])
print("95% CI L1:", results_L1["ci_lower"], "-", results_L1["ci_upper"])
```

```
Mean Bootstrap AUC: 0.9380363535433515
95% CI: 0.9334129724123207 - 0.9426364958302376
Mean Bootstrap L1 AUC: 0.952286596500134
95% CI L1: 0.9485308350614472 - 0.9560324634048928
```

- With 10-fold cross-validation, the accuracies between each fold vary between 82.2% and 85.2% for the default model, 86.6% and 88.7% for removing age and emp_exp, and 89% and 90.2% for L1-regularization. There is a strict increase in accuracy by removing extraneous features.
- The 95% AUC confidence intervals for removing age and emp_exp and L1 regularization are [93.34, 94.26]% and [94.85, 95.60]%, respectively. For example, that is to say that we are 95% confident that a randomly selected sample will have an AUC between 93.34% and 94.26% by removing the age and emp_exp features from the dataset. Also, the L1-regularization model has a tighter interval, indicating higher precision.

4.2 Linear Discriminant Analysis

Linear Discriminant Analysis imposes some assumptions about the dataset, which must be considered before proceeding:

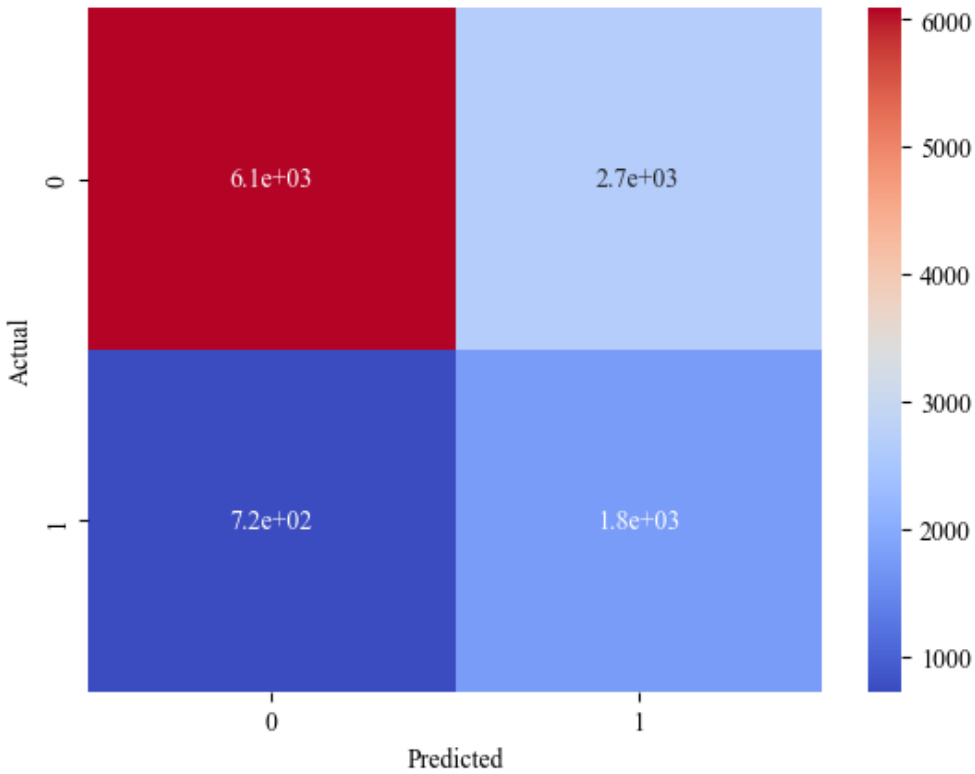
- Features follow a Gaussian (normal) distribution within each class.
- Observations are independent of each other
- The covariance matrix is the same for each class.

The second is assumed to be satisfied (borrowers are independent of each other), the third will be done through the LDA submodule in Scikit-Learn, leaving us to just standardize the features.

[29]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
scaler = StandardScaler()

# Standardizing features
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Fitting a LDA object to our dataset
lda = LinearDiscriminantAnalysis()
lda_model = lda.fit(X_train, y_train)

# Covariance matrix
lda_pred = lda.predict(X_test_scaled)
lda_prob = lda.predict_proba(X_test_scaled)[:, 1]

accuracy = accuracy_score(y_test, lda_pred)
print(f"LDA Accuracy: {accuracy:.4f}")

print(metrics.confusion_matrix(y_test, lda_pred))
```

```
LDA Accuracy: 0.6982
[[6089 2674]
 [ 721 1766]]
```

[30]:
```
sns.heatmap(metrics.confusion_matrix(y_test, lda_pred), annot = True, cmap =
 ↪'coolwarm')
plt.xlabel('Predicted')
plt.ylabel('Actual');
```



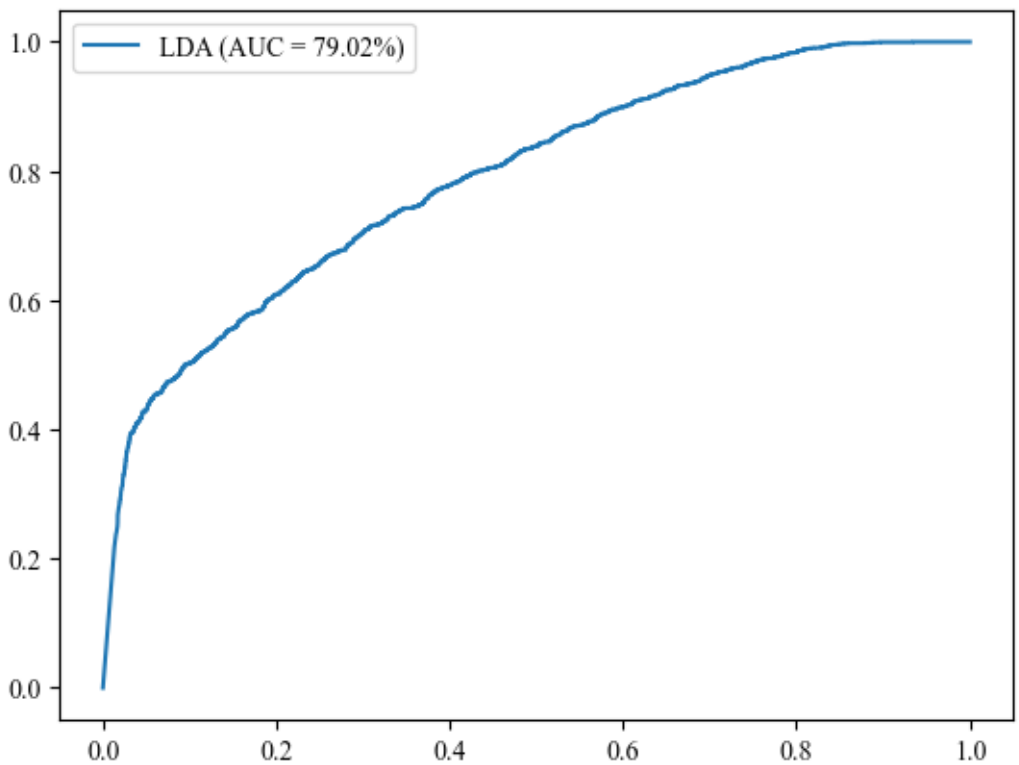
[31]:
```
# Classification
print(metrics.classification_report(y_test, lda_pred, target_names = ['Default',
↪'Paid on time']))
```

```
              precision    recall  f1-score   support

     Default       0.89      0.69      0.78      8763
Paid on time       0.40      0.71      0.51      2487

    accuracy                           0.70     11250
   macro avg       0.65      0.70      0.65     11250
weighted avg       0.78      0.70      0.72     11250
```

The F1-score significantly drops for both target classes, with a more noticable decrease in the paid-on-time class. Additionally, there are considerably more type I than type II errors, meaning the model has a tougher time predicting borrowers who repaid their loan that actually do so (roughly 40% precision on the test data), suggesting that the LDA model frequently misclassifies defaulters as reliable borrowers. Such errors are costly in credit risk applications; they correspond to approving loans for higher-risk borrowers. Since LDA assumes normally distributed features with equal covariance structures among classes, violations of these assumptions in the data may contribute to the lowered classification performance (i.e. credit data tend to have nonlinear boundaries and skewed features). This, combined with a class imbalance, weakens LDA's overall performance greatly.

[32]:
```
# ROC Curve for LDA
fprLDA, tprLDA, _ = metrics.roc_curve(y_test, lda_prob)
auc = metrics.roc_auc_score(y_test, lda_prob)

plt.plot(fprLDA, tprLDA, label = f"LDA (AUC = {auc:.2%})")
plt.legend()
```

[32]: <matplotlib.legend.Legend at 0x1d9dfa15a60>



There is also a drop in the AUC compared to the logistic models, but an AUC of 79.02% is still reasonably decent discrimination. This is to say, the LDA model can correctly rank a randomly chosen repaid loan above a defaulted loan approximately 79% of the time. From sensitivity levels of 0 to 0.4, the model is separating the easiest cases of borrowers, capturing a large ratio of true positives over false positives at lower thresholds. Once the sensitivity reaches 0.4, the slope begins to flatten, implying that increasing sensitivity requires accepting more false positive and that the model has already captured the easiest borrowers to separate. That being said, approving loans in the 40% low risk group adds very little default risk, but when faced with similar risk profiles for both default and repayment classes, the model struggles to differentiate them cleanly. Therefore, approving loans beyond a sensitivity level of 0.4 would increase the probability of lending to borrowers who will ultiamtely default. This is possibly a byproduct of LDA producing only linear decision boundaries, which is a notable limitation if interactions between borrower characteristics and default risk cannot be described linearly.

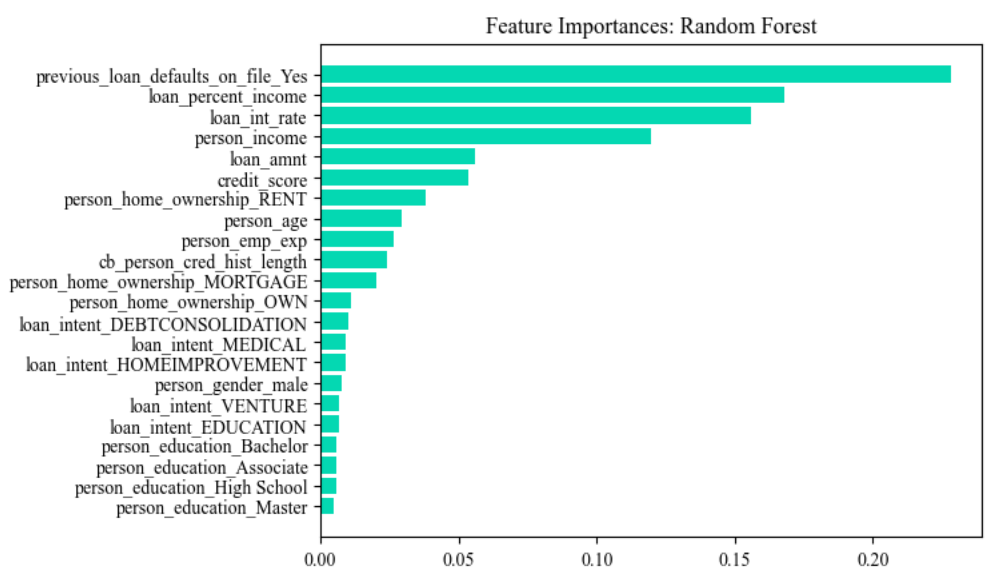
4.3 Ensemble Methods

Lastly, we will look at ensemble methods such as random forests and boosting and compare their performance to Logistic Regression and LDA models.

4.3.1 Random Forest

The fitted random forest classification model will use 1000 decision trees. We will set the max features m per tree split using $m \approx \sqrt{p}$, or $m \approx 5$ in this case. So, at each split, 5 features will be randomly sampled.

[33]:
```
# Random Forest Classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

forest = RandomForestClassifier(n_estimators = 1000, max_features = 5)
forest.fit(X_train, y_train)

# Ranking Feature Importances
importances = forest.feature_importances_
indices = np.argsort(importances)
plt.barh(X.columns[indices], importances[indices], color = "#04D8B2")
plt.title("Feature Importances: Random Forest");
plt.show()

# Computing Accuracy
forest_test = forest.predict(X_test)
forest_score = accuracy_score(y_test, forest_test)
print(f'Random Forest Test Data Accuracy = {forest_score:.4f}')
```




```
Random Forest Test Data Accuracy = 0.9287
```

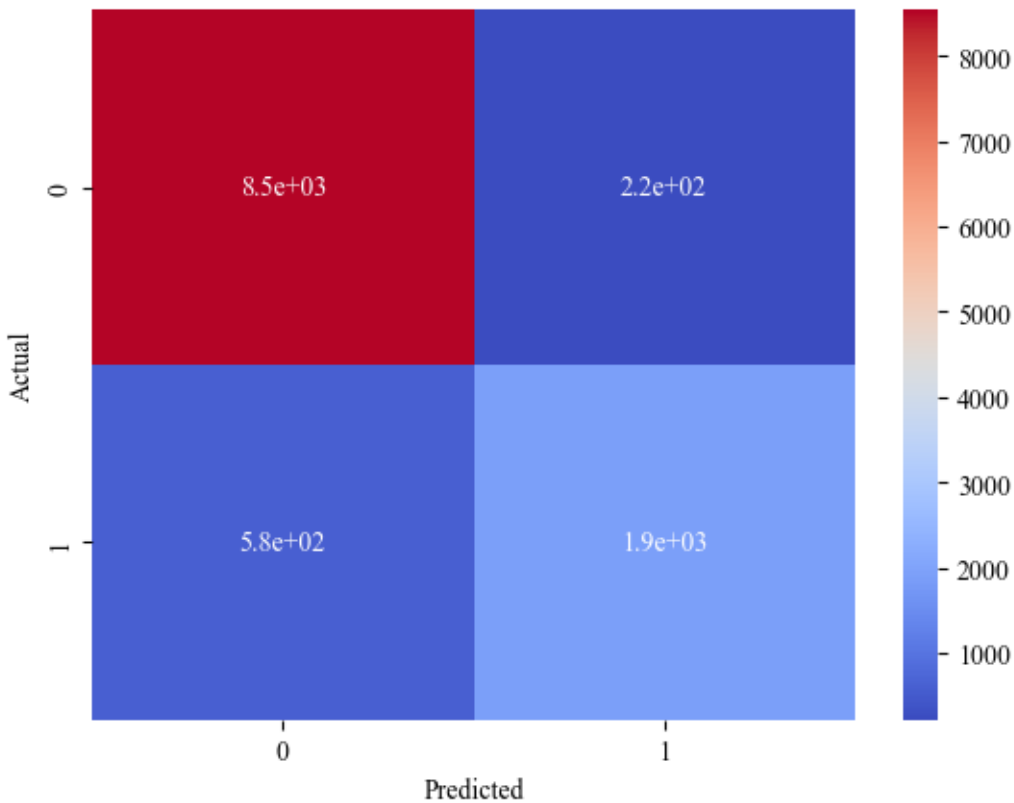
Compared to the coefficient magnitudes in the L1 logistic regression model, there are similarities in the highest ranking feature importances. Notably, 4 out of the top 5 features, by importance, are the same, with

an exact ranking match in the top 3. Also, random forest asserts a more conservative estimate on previous defaults by giving more weight to loan income ratio, loan interest rate, and annual income. While this has led to an increase in accuracy, classification and AUC must be tested to verify random forest's improvement over logistic regression.

[34]:
```
# Confusion Matrix

y_predRF = forest.predict(X_test)
y_probaRF = forest.predict_proba(X_test)[:, 1]

print(metrics.confusion_matrix(y_test, y_predRF))
sns.heatmap(metrics.confusion_matrix(y_test, y_predRF), annot = True, cmap =
 ↪'coolwarm')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

```
[[8542  221]
 [ 581 1906]]
```

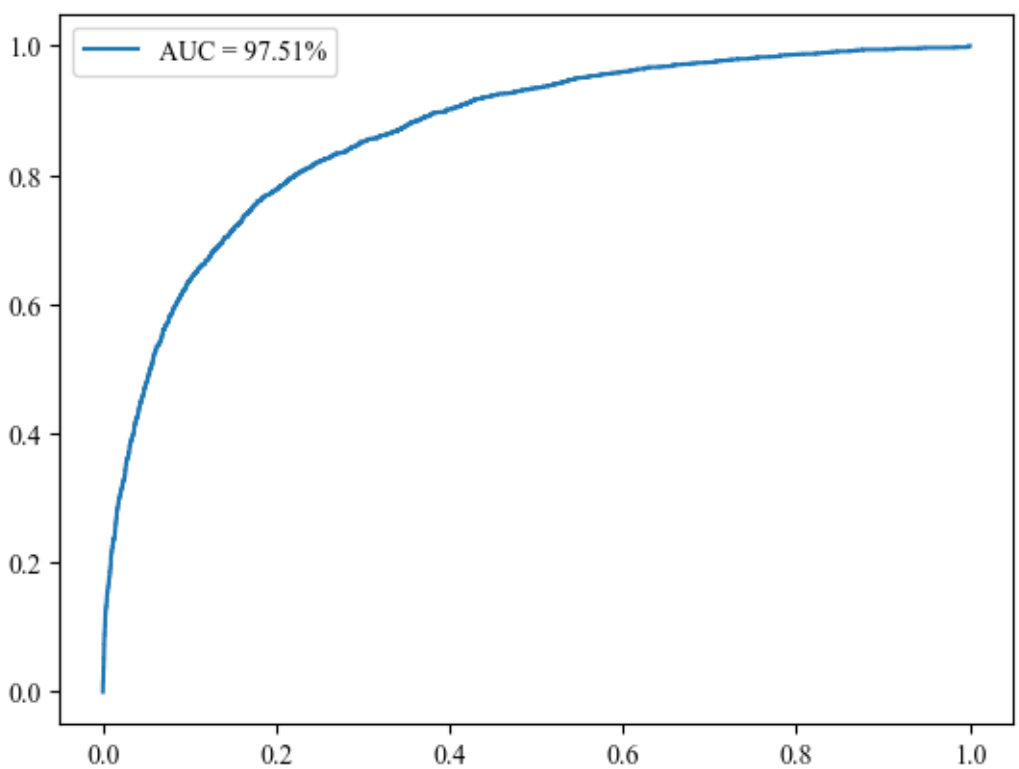
[34]: Text(50.722222222222214, 0.5, 'Actual')



[35]:
```
# Classification report

print(metrics.classification_report(y_test, y_predRF, target_names=['Default',
 ↪'Paid on time']))
```

```
              precision    recall  f1-score   support

     Default       0.94      0.97      0.96      8763
Paid on time       0.90      0.77      0.83      2487

    accuracy                           0.93     11250
   macro avg       0.92      0.87      0.89     11250
weighted avg       0.93      0.93      0.93     11250
```

[36]:
```
fprRF, tprRF, _ = metrics.roc_curve(y_test, y_probaRF)
auc = metrics.roc_auc_score(y_test, y_probaRF)
plt.plot(fpr, tpr, label=f"AUC = {auc:.2%}")
plt.legend();
```



[37]:
```
results_RF = bootstrap_auc(y_test, y_probaRF)
print("Mean Bootstrap Random Forest AUC:", results_RF["mean_auc"])
```

```
print("95% CI Random Forest:", results_RF["ci_lower"], "-", results_RF["ci_upper"])
```

```
Mean Bootstrap Random Forest AUC: 0.9751393431207084
95% CI Random Forest: 0.9726832213228929 - 0.9777846629372879
```

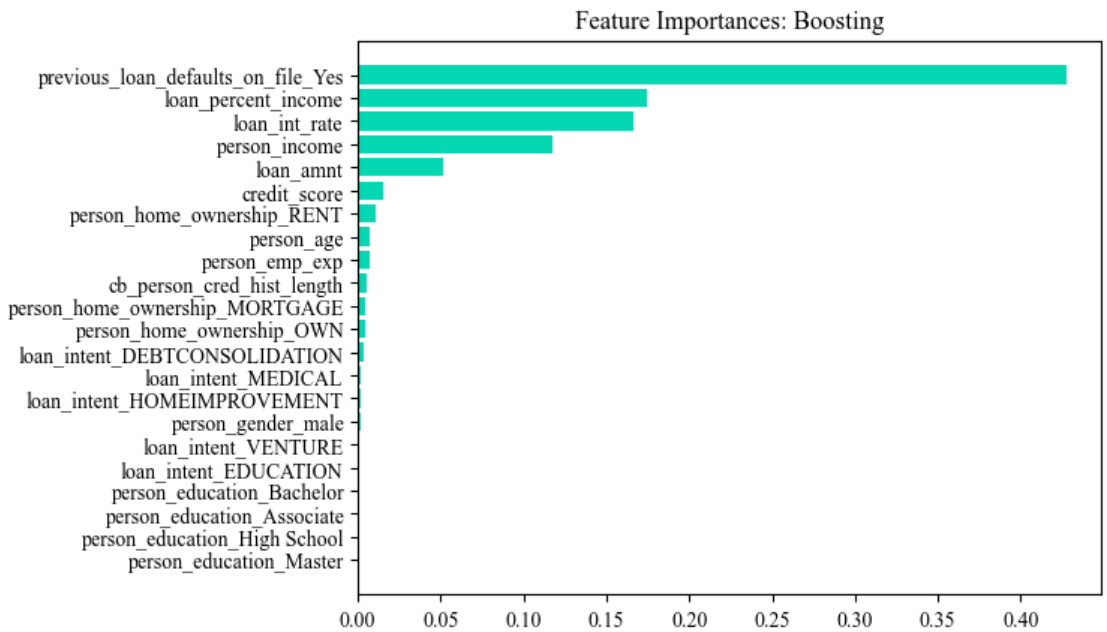
Random forest delivers a significant increase in predictability and separability strength as shown by its accuracy, F1-score, and AUC. The total amount of type I/II errors are both lower compared to its predecessors. Only 223 false positives were recordered, giving Random Forest the lowest false-positive rate yet. The modest increase in F1-score in both classes makes Random Forest the best classifier on the test data so far. Lastly, the ROC curve follows a consistent trajectory as probability thresholds increase, indicating a high degree of separability.

4.3.2 Boosting

The last model to discuss is boosting, which iteratively improves model performance by training "weaker" trees sequentially to correct misclassifications of previous trees. In boosting, a learning rate λ (typically small) is introduced to control the rate at which the algorithm's parameters are adjusted at each tree.

In our algorithm, we impose Gradient Boosting Classification with 1000 trees, a maximum depth of 3, and $\lambda = 0.1$.

[38]:
```
# Boosting model
from sklearn.ensemble import GradientBoostingClassifier

gbRF = GradientBoostingClassifier(n_estimators = 1000, learning_rate = 0.1,
                                  max_depth = 3, random_state = 42)
gbRF.fit(X_train, y_train)

#Ranking Feature Importances
importances_boost = gbRF.feature_importances_
indices_boost = np.argsort(importances_boost)
plt.barh(X.columns[indices], importances_boost[indices_boost], color = "#04D8B2")
plt.title("Feature Importances: Boosting");
plt.show()

#Computing Accuracy
boost_test = gbRF.predict(X_test)
boost_score = accuracy_score(y_test, boost_test)
print(f'Boosting Test Data Accuracy = {boost_score:.4f}')
```



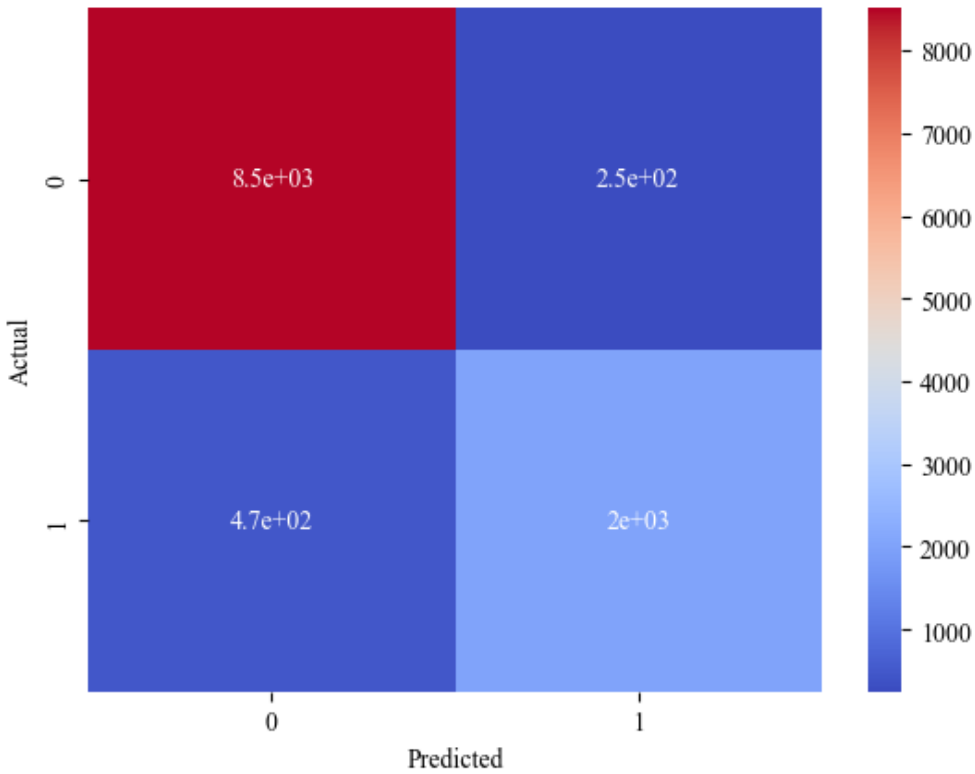

```
Boosting Test Data Accuracy = 0.9362
```

Compared to the conservative approximations of random forest, boosting concentrates high importance on our most dominant features. In our Random Forest model, each feature had more share in their importance in predicting a borrower's probability to default. Now, more than 90% are split between 5 features: previous defaults, loan:income ratio, loan interest rate, annual income, and loan amount, with previous defaults recorded as the governing feature at over 40% importance. Despite this aggressive philosophy and noticeable difference in feature importances, boosting supersedes Random Forests in accuracy by only 0.7%. Depending on boosting's F1-score and AUC, the boosting algorithm may solidify the top features that can help lenders decide if they should approve a requested loan.

[39]:
```
# Confusion Matrix

y_pred_boost = gbRF.predict(X_test)
y_prob_boost = gbRF.predict_proba(X_test)[:, 1]

print(metrics.confusion_matrix(y_test, y_pred_boost))
sns.heatmap(metrics.confusion_matrix(y_test, y_pred_boost), annot = True, cmap =
 ↪'coolwarm')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

```
[[8514  249]
 [ 469 2018]]
```

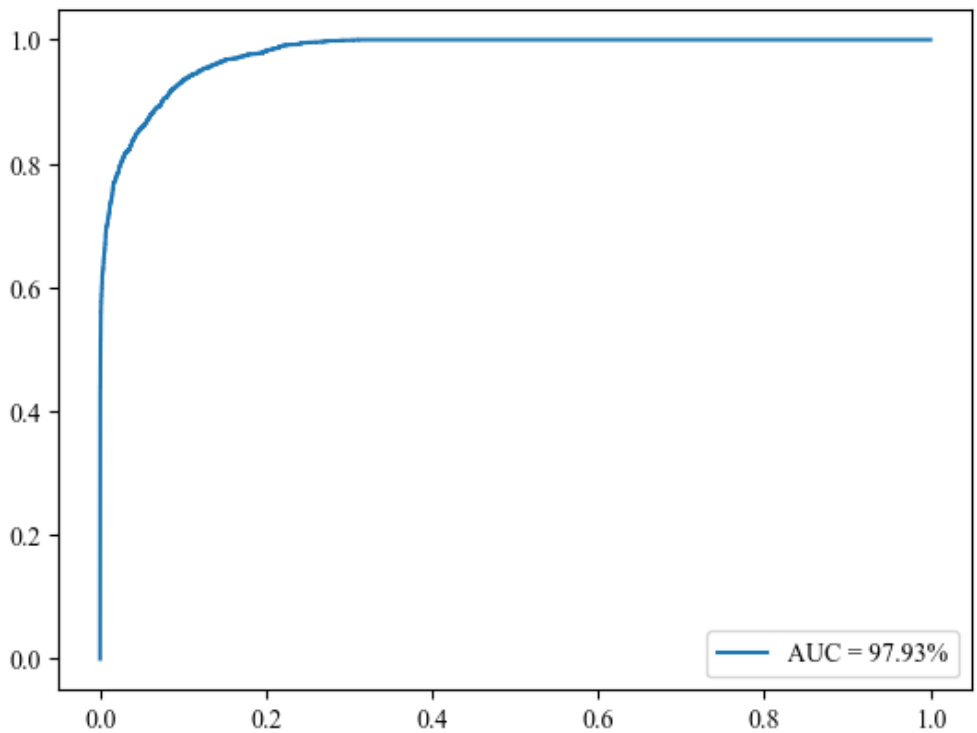
[39]: Text(50.722222222222214, 0.5, 'Actual')



[40]:
```
# Classification report

print(metrics.classification_report(y_test, y_pred_boost, target_names =
 ↪['Default', 'Paid on time']))
```

```
              precision    recall  f1-score   support

     Default       0.95      0.97      0.96      8763
Paid on time       0.89      0.81      0.85      2487

    accuracy                           0.94     11250
   macro avg       0.92      0.89      0.90     11250
weighted avg       0.94      0.94      0.94     11250
```

[41]:
```
# ROC Curve
fpr_boost, tpr_boost, _ = metrics.roc_curve(y_test, y_prob_boost)
auc = metrics.roc_auc_score(y_test, y_prob_boost)
plt.plot(fpr_boost, tpr_boost, label=f"AUC = {auc:.2%}")
plt.legend();
```



[42]:
```
# 95% CI
results_boost = bootstrap_auc(y_test, y_prob_boost)
print("Mean Bootstrap Boosting AUC:", results_boost["mean_auc"])
print("95% CI Boosting:", results_boost["ci_lower"], "-", results_boost["ci_upper"])
```

```
Mean Bootstrap Boosting AUC: 0.9793094722359543
95% CI Boosting: 0.977232814630035 - 0.9814622229098463
```

Although the F1-score for defaulting is practically the same as the Random Forest model, the F1-score for paid-on-time increased by 2%, implying that this class has improved predictability performance and better identification. Even though precision is slightly lower for paid-on-time, the jump in recall results in a net F1-score gain. Moreover, the AUC of 97.93% signals very strong separability, with some minor misclassifications at the 0.8 sensitivity level. Therefore, the model has an easy time separating most of the test data and only begins accepting false positives when the probability threshold is between 0.8 and 1.

4.4 Summary

Below is a summary of the key metrics for Modified Logistic, L1-regularization (Lasso), LDA, Random Forest, and Boosting. Accuracy will use the 10-fold CV mean for the logistic models.

	Modified Logistic	Lasso Model	LDA	Random Forest	Boosting
Accuracy	0.8768	0.8968	0.6982	0.9288	0.9362
Default F1	0.9200	0.9300	0.7800	0.9600	0.9600
Repaid F1	0.7300	0.7600	0.5100	0.8300	0.8500
Repaid Precision	0.7400	0.7700	0.4000	0.9000	0.8900
AUC	0.9380	0.9522	0.7902	0.9751	0.9793

4.5 Conclusion

After extensive analyses of six models, the Boosting model promises the best performance and probability of borrower default, for it offers superior accuracy, predictability, and separability against all others. Even though the default class is expected to have easier separability/predictability, the main concern of differentiating the minority class, repaying on time, is best resolved with boosting, achieving a remarkable F1-score of 0.85. Therefore, the boosting model offers the lowest default risk and will minimize the loss incurred from borrower default, and all risk computations will use said model moving forward.

5 Amortization Modeling

In this section, fixed income techniques such as amortization modeling will be employed to simulate a repayment schedule for each borrower in the dataset. Of which, annuity payments will be calculated for future sections. Additionally, a detailed amortization schedule will be provided that can be applied to any borrower.

First, we will construct a new dataset from existing columns while appending the probability of default as a new column.

[43]:
```
probabilities = gbRF.predict_proba(X_scaled)[:, 0]
df_amort = df[['loan_amnt', 'loan_int_rate', 'loan_intent', 'loan_percent_income']]
df_amort["default_prob"] = probabilities
```

The original dataset does not include the loan's term, so assumptions need to be set in place. - In our case, each class of loan intent will be assigned its own loan term. For example, all loans requested to pay off student loans would fall under the education class, thereby making the loan term 180 months. This offers a slightly more dynamic method of amortizing loans as opposed to fixing a single term for each borrower. The dictionary of loan terms will be mapped as a new column. - Additionally, we assume that loans are repaid in monthly installments. Therefore, annual rates will be converted into an effective rate per month, also concatenated as a new column. - Lastly, for a future calculation, the borrower's monthly income is required. We assume the amount earned each month is static, equalling the annual income divided by 12.

[44]:
```
# Mapping loan_intent to a numerical length term
loan_term_map = {
    "PERSONAL": 60,
    "EDUCATION": 180,
    "MEDICAL": 60,
    "VENTURE": 48,
    "HOMEIMPROVEMENT": 36,
    "DEBTCONSOLIDATION": 48
}

df_amort["loan_term_length"] = df_amort["loan_intent"].map(loan_term_map)
df_amort["annual_rate"] = df_amort["loan_int_rate"]/100
df_amort["monthly_rate"] = (df_amort["annual_rate"] + 1)**(1/12) - 1
df_amort["monthly_income"] = df["person_income"]/12
df_amort
```

[44]:
```
        loan_amnt  loan_int_rate        loan_intent  loan_percent_income  \
0         35000.0          16.02           PERSONAL                 0.49
1          1000.0          11.14          EDUCATION                 0.08
2          5500.0          12.87            MEDICAL                 0.44
3         35000.0          15.23            MEDICAL                 0.44
4         35000.0          14.27            MEDICAL                 0.53
…            …             …                 …                    …
44995     15000.0          15.66            MEDICAL                 0.31
44996      9000.0          14.07    HOMEIMPROVEMENT                 0.14
44997      2771.0          10.02  DEBTCONSOLIDATION                 0.05
```

```
44998    12000.0          13.23          EDUCATION                 0.36
44999     6665.0          17.05  DEBTCONSOLIDATION                 0.13

       default_prob  loan_term_length  annual_rate  monthly_rate  \
0          0.000002                60       0.1602      0.012460
1          0.999888               180       0.1114      0.008841
2          0.000029                60       0.1287      0.010140
3          0.000002                60       0.1523      0.011883
4          0.000002                60       0.1427      0.011178
…               …                 …            …            …
44995      0.000003                60       0.1566      0.012198
44996      0.001100                36       0.1407      0.011031
44997      0.278066                48       0.1002      0.007989
44998      0.000002               180       0.1323      0.010408
44999      0.256756                48       0.1705      0.013206

       monthly_income
0         5995.666667
1         1023.500000
2         1036.500000
3         6646.083333
4         5511.250000
…                 …
44995     3997.583333
44996     5483.333333
44997     4745.166667
44998     2763.666667
44999     4300.750000

[45000 rows x 9 columns]
```

We can now express the annuity payments R using the given information. Let i_k, n_k, and L_k represent the monthly interest rate, term length, and loan amount for the k-th borrower. Then

$$R = \frac{L_k i_k}{1 - (1 + i_k)^{-n_k}}$$

[45]:
```
# Annuity payment calculation
df_amort["monthly_payment"] = (
    df_amort["loan_amnt"] * df_amort["monthly_rate"]) / (1 - (1 +
 ↪df_amort["monthly_rate"]) ** (-df_amort["loan_term_length"]))
df_amort[["loan_amnt", "loan_term_length", "monthly_income", "monthly_payment"]]
```

[45]:
```
       loan_amnt  loan_term_length  monthly_income  monthly_payment
0        35000.0                60     5995.666667       831.758985
1         1000.0               180     1023.500000        11.121452
2         5500.0                60     1036.500000       122.811707
3        35000.0                60     6646.083333       819.115301
```

```
4        35000.0                60     5511.250000       803.791255
…          …                  …          …                …
44995    15000.0                60     3997.583333       353.997300
44996     9000.0                36     5483.333333       304.272595
44997     2771.0                48     4745.166667        69.731782
44998    12000.0               180     2763.666667       147.821795
44999     6665.0                48     4300.750000       188.365511

[45000 rows x 4 columns]
```

Another notable property of loan amortization is **interest**, or the total amount repaid as a percentage of the outstanding balance before each payment. Let $\sum I_k$ be the total interest for the k-th borrower, then

$$\sum I_k = R_k n_k - L_k$$

...assuming level payments throughout the duration of the loan, which applies to our data by construction. We say $R_k n_k$ is the repayment amount, or the total raw amount repaid.

A brief analysis will be conducted on the monthly payments and total interest paid over each loan intent class.

[46]:
```python
# Repayment amount and total interest paid
df_amort["repayment_amount"] = df_amort["monthly_payment"] *
 ↪df_amort["loan_term_length"]
df_amort["total_interest"] = df_amort["repayment_amount"] - df_amort["loan_amnt"]

# Statistics of total interest by each loan intent class
df_amort.groupby("loan_intent")["total_interest"].describe()
```

[46]:
```
                     count         mean          std         min          25%  \
loan_intent
DEBTCONSOLIDATION   7145.0  2250.118419  1815.936848  110.464985   966.788367
EDUCATION           9153.0  9601.865922  7734.734267  450.753650  4318.036559
HOMEIMPROVEMENT     4783.0  1859.853150  1453.998450   75.321934   809.235311
MEDICAL             8548.0  2741.734223  2244.714590  150.082693  1218.600748
PERSONAL            7552.0  2845.240142  2246.751963  137.259005  1271.505107
VENTURE             7819.0  2246.766139  1738.238058  111.719391  1003.095841

                         50%           75%           max
loan_intent
DEBTCONSOLIDATION  1720.498416   2975.999848  14676.607943
EDUCATION          7313.967362  12589.374365  65411.238612
HOMEIMPROVEMENT    1425.075325   2496.556027  10787.093310
MEDICAL            2075.453532   3500.615729  18751.350134
PERSONAL           2181.578800   3706.534302  17433.100448
VENTURE            1718.196444   3009.318935  12807.246881
```

Education having the highest total interest per quartile is unsurprising because it has the longest term among all six classes. As a result, the interest per payment decreases at a slower rate and leads to more moderately sized interest amounts. On the contrary, home improvement has the lowest total interest

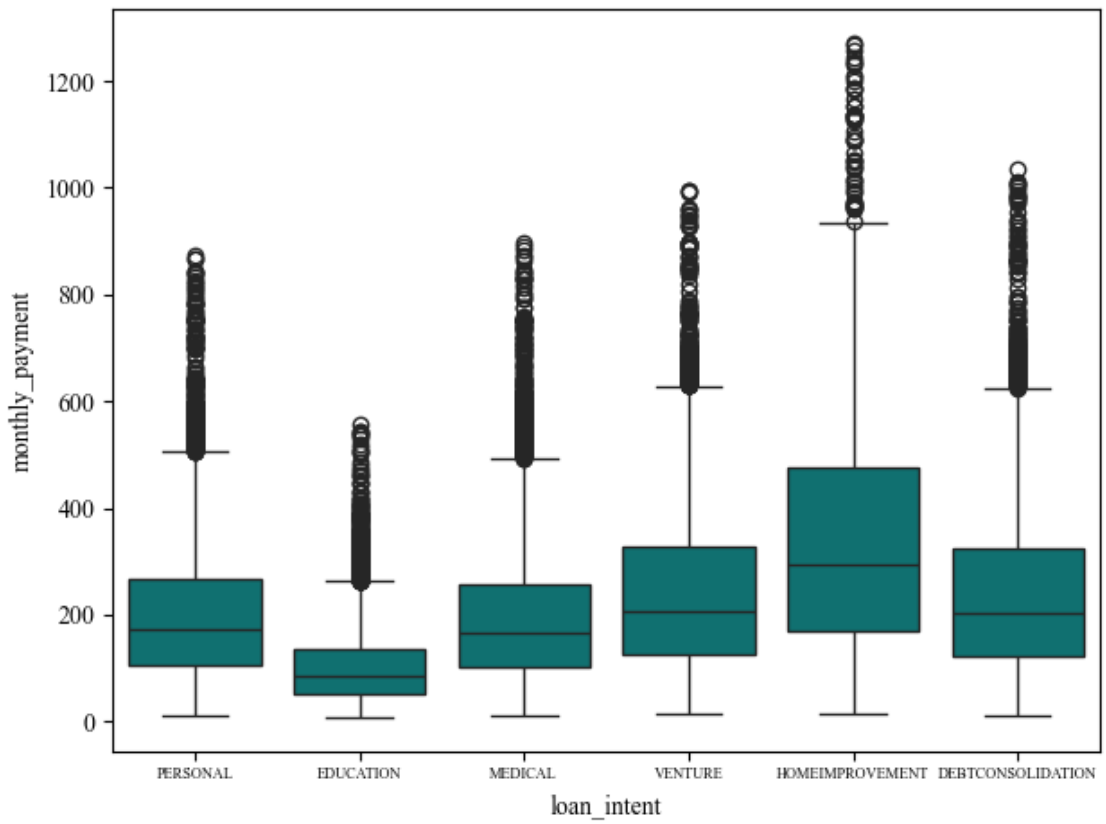
because it has the shortest term among all six classes. The order, using the quartiles, from largest to smallest total interest is:

Education > Medical > Personal > Debt Consolidation > Venture > Home Improvement

[47]:
```
# Statistics of monthly payment by each loan intent class
df_amort.groupby("loan_intent")["monthly_payment"].describe()
```

[47]:
```
                     count        mean         std        min         25%  \
loan_intent
DEBTCONSOLIDATION  7145.0  245.801085  168.085031  12.799420  122.874433
EDUCATION          9153.0  106.162622   75.637916   8.059743   53.364044
HOMEIMPROVEMENT    4783.0  341.084458  222.551458  15.981165  168.425897
MEDICAL            8548.0  197.994890  137.199991  10.834712  101.346828
PERSONAL           7552.0  207.485976  139.912432  10.620983  106.928852
VENTURE            7819.0  248.034658  164.669947  14.422029  126.612816

                         50%         75%          max
loan_intent
DEBTCONSOLIDATION  204.158959  323.870659  1034.929332
EDUCATION           86.945505  137.337850   557.840215
HOMEIMPROVEMENT    294.679743  476.059135  1271.863703
MEDICAL            165.849826  257.870929   895.855836
PERSONAL           174.109631  266.325753   873.885007
VENTURE            208.233993  327.629254   995.984310
```

[48]:
```
# Boxplot on monthly payment by loan intent
bplot = sns.boxplot(data = df_amort, x = "loan_intent", y = "monthly_payment", color
 ↪= "teal")
bplot.set_xticklabels(bplot.get_xticklabels(), fontsize = 6)
plt.tight_layout()
```



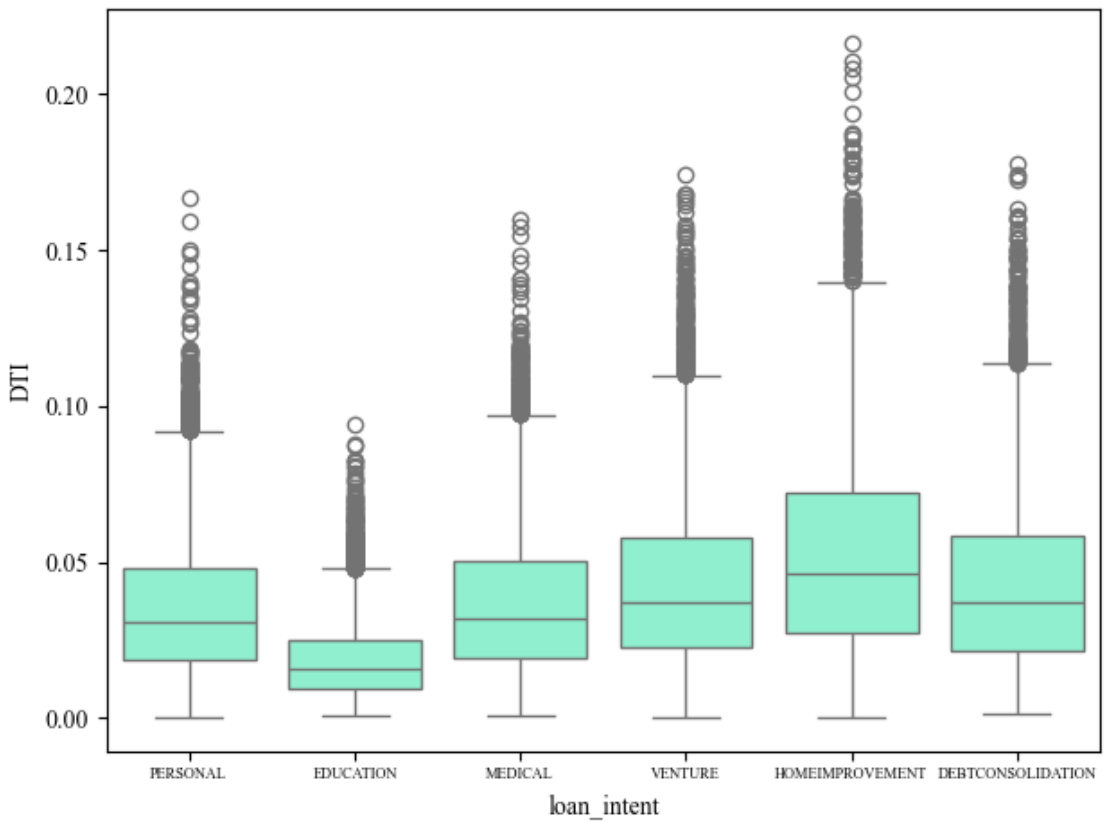
The monthly payment statistics reveal that although education has the largest total interest, it has the lowest monthly payment between all six classes and home improvement having the highest. Of course, these results are mostly contingent on the loan term; as a loan's term increases, the resulting sum of interest increases and the monthly payment decreases. The loan amount for each borrower also contributes to the statistic; some are larger than others, thereby not resulting in a completely inverted order from total interest:

Home Improvement > Debt Consolidation > Venture > Medical > Personal > Education

It is also worth noting that the mean monthly payment per class do not differ as much compared to their maximums.

Another measurement of interest is a borrower's monthly **debt-to-income (DTI)** ratio, or the percentage of one's monthly income used toward monthly loan payments. The lower a borrower's DTI is, the more likely it is for a lender to approve a loan (typically no more than 30%-40%), but the percentage range varies by intent. For simplicity, we assume that the single loan payment is the only debt expended by the borrower each month.

[49]:
```
# Boxplot on monthly-debt-to-income (DTI) ratio
df_amort["DTI"] = df_amort["monthly_payment"]/df_amort["monthly_income"]
DTIplot = sns.boxplot(data = df_amort, x = "loan_intent", y = "DTI", color =
 ↪"aquamarine")
DTIplot.set_xticklabels(DTIplot.get_xticklabels(), fontsize = 6)
plt.tight_layout()
```



The DTI boxplots are similar in structure compared to the monthly payments. The order is almost the same, but the quartiles for personal are slightly lower than medical. More importantly, the maximum DTI ratio is ~ 0.22; most lenders would approve every borrower, but again we must consider the underlying assumption made earlier. If borrowers had more repayments to be made each month, some loans would not get approved.

Before approving a loan, lenders typically generate a final amortization schedule to confirm the borrower's ability to successfully repay the loan in full. Interest and principal breakdowns are also performed to track equity growth and allow the borrower to determine how much they can save by shortening the term.

Below is an implementation of an amortization schedule, which will then be applied to a random borrower in the dataset.

[50]:
```
# Full amortization schedule

def amortization_schedule(loan_amount, monthly_rate, term_length, payment):
    """ Returns an amortization schedule of a loan
        Arguments: loan amount, monthly interest rate, term length, monthly payment
        Output: A pandas DataFrame with a term_length x 5 shape
        The columns include the payment, interest, principal, and remaining balance
        for each month
    """

    balance = loan_amount
```

```
    schedule = []

    for month in range(1, term_length + 1):  # Iterates over each month in loan term
        interest = balance * monthly_rate
        principal = payment - interest
        balance -= principal

        # Add columns to the schedule DataFrame
        schedule.append({
            "Month": month,
            "Payment": payment,
            "Interest": interest,
            "Principal": principal,
            "Remaining Balance": max(balance, 0)
        })

    return pd.DataFrame(schedule)

# Applying to a randomly chosen observation
ob1 = df_amort.sample().iloc[0]
ob1
```

[50]:
```
loan_amnt                   2500.0
loan_int_rate                 11.3
loan_intent               PERSONAL
loan_percent_income           0.04
default_prob              0.999979
loan_term_length                60
annual_rate                  0.113
monthly_rate              0.008962
monthly_income         5545.666667
monthly_payment          54.049614
repayment_amount       3242.976854
total_interest          742.976854
DTI                       0.009746
Name: 35783, dtype: object
```

[51]:
```
ex_amort = amortization_schedule(
    ob1['loan_amnt'], ob1['monthly_rate'], ob1['loan_term_length'],
 ↪ob1['monthly_payment'])
ex_amort.head(5)
```

[51]:

	Month	Payment	Interest	Principal	Remaining Balance
0	1	54.049614	22.403763	31.645851	2468.354149
1	2	54.049614	22.120169	31.929445	2436.424704
2	3	54.049614	21.834033	32.215581	2404.209123
3	4	54.049614	21.545333	32.504281	2371.704841

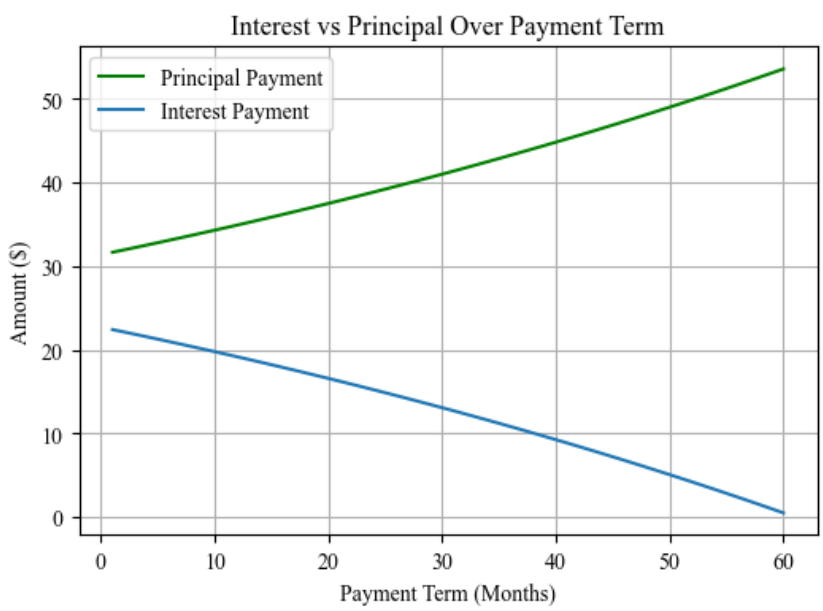
```
4      5  54.049614  21.254046  32.795569        2338.909273
```

[52]:
```
ex_amort.tail(5)
```

[52]:

	Month	Payment	Interest	Principal	Remaining Balance
55	56	54.049614	2.358057	51.691557	2.114403e+02
56	57	54.049614	1.894823	52.154791	1.592855e+02
57	58	54.049614	1.427438	52.622177	1.066633e+02
58	59	54.049614	0.955864	53.093750	5.356955e+01
59	60	54.049614	0.480064	53.569550	1.243450e-12

[53]:
```
# Interest vs. Principal over term length
plt.figure(figsize = (6, 4))
plt.plot(ex_amort['Month'], ex_amort['Principal'], label = 'Principal Payment',
 ↪color = "green")
plt.plot(ex_amort['Month'], ex_amort['Interest'], label = 'Interest Payment')
plt.xlabel('Payment Term (Months)')
plt.ylabel('Amount ($)')
plt.title('Interest vs Principal Over Payment Term')
plt.legend()
plt.grid(True)
plt.show()
```



Because the remaining balance after 60 months is zero and the principal grows exponentially, the amortization schedule function is working as intended.

6 Default Risk Measurement

Moving forward from amortization schedules, we wish to build upon this foundation by identifying the expected cash flow, or the amount a borrower is expected to repay, adjusting for the probability that they

default at each payment in the amortization term. In the models/methodology section, we established a boosting model to estimate the probability a borrower will default. However, this probability estimation is given at the start of the amortization term and remains fixed throughout. Let CF and PD represent the array of annuity installments and the probability that the borrower defaults, respectively. We could calculate the expected loan value using a single fixed probability:

$$E[CF] = (1 - PD) \times \sum_{t=1}^{n} \frac{R_t}{(1+i)^t}$$

...but it fails to account for real-world circumstances or events that affect the borrower. We can instead make the probability dynamic using a survival probability, or the probability that the loan will not be terminated before a certain time. It is a function of the hazard rate, denoted by λ, which is the instantaneous probability of default at time t given that the loan has survived up to t. Finally, as time progresses, the survival rate decreases. We will aim to model a constant hazard rate using the probability assigned to each borrower. Fix

$$\lambda = -\frac{\ln(1 - PD)}{n} \in [0, 1)$$

(obtained by rearranging the exponential CDF). Then λ will increase as PD increases. Let $S(t) = e^{-\lambda t}$, then $S(t)$ decreases and

$$E[L] = \sum_{t=1}^{n} \frac{S(t) \cdot R_t}{(1+i)^t}$$

will reflect the expected loan value for each borrower, assuming the borrower survives to time t. This way, the survival probability will change after each payment and have a better representation of real-world situations.

[54]:
```
# Hazard rate, survival probability, expected loan value computations

def hazard_rate(pd, term):
    """ Returns the constant hazard rate of a borrower's ability to
        repay a loan
        Arguments: probability of default and term of the loan
        Output: the constant hazard rate expressed as a real number
    """
    return -np.log(1 - pd)/term

def survival_prob(pd, term):
    """ Returns the survival probabilities at each payment in
        a loan amortization term for a given borrower
        Arguments: probability of default and term of the loan
        Output: a 1 x term array containing survival probabilities,
        each expressed as real numbers
    """
    lamb = hazard_rate(pd, term)
    t = np.arange(1, term + 1)
    Sp = np.exp(-lamb * t)
    return t, Sp
```

```python
def expected_LV(pd, term, R, i):
    """ Returns the expected loan value for a given borrower
        Arguments: probability of default, term of the loan,
        monthly payments, and monthly interest rate
        Output: the expected loan value, expressed as a
        real number
    """
    t, Sp = survival_prob(pd, term)
    v_t = 1/(1 + i)**t
    return np.sum(Sp * R * v_t)


test_LV = expected_LV(
    ob1['default_prob'], ob1['loan_term_length'], ob1['monthly_payment'],
 ↪ob1['monthly_rate'])
test_LV
```

[54]: 260.7360608961678

Recall that for this borrower, the actual loan amount is 2,500 and their default of probability is basically 100% (0.999979). Therefore, the survival probability will be decreasing an at incredibly accelerated rate and will greatly reduce the expected loan value, which is generally a warning sign for the lender. In this case, the lender expects to lose 2,239.26, from which they would most likely deny a loan request from this borrower.

Knowing the expected loan value and loan amount will give us the expected return for lenders, or the percentage difference between the expected loan amount and the true loan amount.

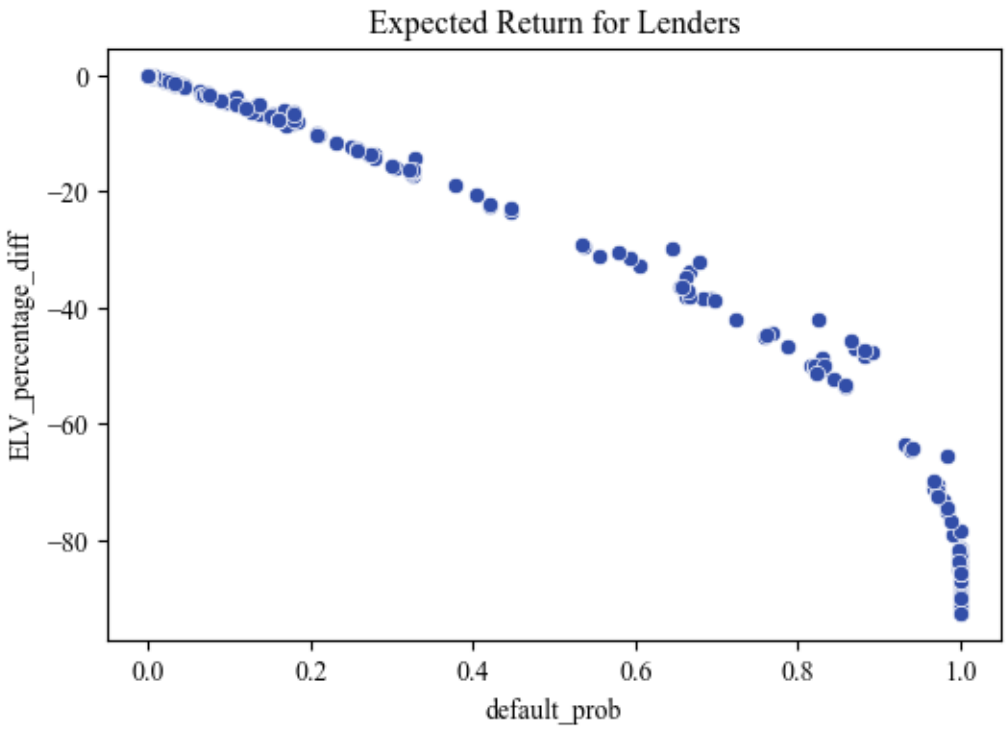
[55]:
```python
# Adding expected loan value column
ELV = pd.DataFrame(np.zeros(len(df_amort)), columns = ['exp_LV'])

for i in range(len(df_amort)):
    ob = df_amort.iloc[i]
    ob_ELV = expected_LV(
        ob['default_prob'], ob['loan_term_length'], ob['monthly_payment'],
 ↪ob['monthly_rate'])
    ELV.iloc[i] = ob_ELV

df_amort2 = pd.concat([df_amort, ELV], axis = 1)

# Measuring expected return for lenders
df_amort2['ELV_percentage_diff'] = (df_amort2['exp_LV'] - df_amort2['loan_amnt'])/
 ↪df_amort2['loan_amnt'] * 100
df_amort2[['loan_amnt', 'default_prob', 'exp_LV', 'ELV_percentage_diff']]
```

[55]:

	loan_amnt	default_prob	exp_LV	ELV_percentage_diff
0	35000.0	0.000002	34999.975472	-0.000070
1	1000.0	0.999888	181.974535	-81.802547

```
2          5500.0      0.000029   5499.926964            -0.001328
3         35000.0      0.000002  34999.974271            -0.000074
4         35000.0      0.000002  34999.965308            -0.000099
…            …            …           …                   …
44995     15000.0      0.000003  14999.982044            -0.000120
44996      9000.0      0.001100   8995.238079            -0.052910
44997      2771.0      0.278066   2381.218515           -14.066456
44998     12000.0      0.000002  11999.992132            -0.000066
44999      6665.0      0.256756   5838.345421           -12.402919

[45000 rows x 4 columns]
```

[56]:
```
risk_sample = df_amort2.sample(frac = 0.01, random_state = 42)
plt.figure(figsize = (6, 4))
sns.scatterplot(data = risk_sample, x = 'default_prob', y = 'ELV_percentage_diff',
 ↪color = '#324EA8')
plt.title("Expected Return for Lenders")
plt.show()
```



The expected loan value decreases monotonically and nonlinearly as predicted default probability increases. The concave shape reflects the survival probability dropping quickly, and conseqeuntly, the expected loan value collapsing. This is why the expected return steepens as *PD* approaches 1. For example, from a lender's perspective, a default probability increase from 10% to 15% is less catastrophic than from 70% to 75%. From the borrower's perspective, improving credit at low risk levels will have little effect on the expected loan value. However, a slight improvement at high-risk levels can significantly boost financial standing and increase the chance that a loan will be approved.

Also, note that the range for expected return is almost never [-1, 0]. This applies to our dataset because the survival function prevents the expected loan value from ever exceeding the true loan amount since $S(t) \leq 1$ for all t. Because discount factors are always less than 1, we get a negative return for each lender. Typically, there are additional features such as funding and recovery rates that will yield in positive and negative returns for lenders, providing easier distinguishment between safe and risky borrowers (i.e. positive return = safe, negative return = risky).

7 Expected Loss Modeling

Understanding returns for lenders are already a key asset in identifying potential risk in loan defaults. However, lenders also wish to estimate anticipated losses across loans. Expected losses are a crucial metric in credit risk analysis and are used by lenders to use forecasting to maintain financial stability and minimize risk in loan portfolios. There are three key components that are used to measure the expected loss of a loan:

1. **Probability of Default (PD)**: This was computed in the Methodology section and further refined in the Expected Loan Value section to dynamically estimate the probability of default at any given payment. To compute the probability of default at time t, or PD_t, we need to recursively take the difference of the adjacent survival probabilities.

2. **Exposure at Default (EAD)**: Quantifies the total value exposed to loss at the time at default. In our case, this is simply just the outstanding balance at time t

3. **Loss Given Default (LGD)**: The percentage of exposure a lender expects to lose if a default occurs. We have no easy way of measuring this, so we will assume the lender expects to recover 40% of the default amount. Therefore, the LGD is 60%.

Before calculating the expected loss directly, we need to develop an outstanding balance function and verify that it works.

[57]:
```
def outstanding_balance(R, i, t, n):
    """ Returns the outstanding balance of a loan
        after t payments
        Arguments: monthly payment, monthly interest rate,
        current payment term, loan term
        Output: the outstanding balance expressed as a real
        number
    """
    v_i = 1/((1 + i)**(n - t))
    OB = (R * (1 - v_i))/i
    time = t
    return time, OB

time, OB_20 = outstanding_balance(
    ob1['monthly_payment'], ob1['monthly_rate'], 20, ob1['loan_term_length'])

print(f"The outstanding balance after {time} months is: {OB_20}")
```

```
The outstanding balance after 20 months is: 1810.189219367892
```

We need to use the probability of default at t, requiring us to calculate that recursively:

$$PD_t = S(t-1) - S(t)$$

The expected loss function will be calculated on the random sample, and then concatenated as a column to the amortization DataFrame.

[58]:
```
def expected_loss(R, i, n, pd):
    """ Returns the expected loss of a loan
        Arguments: monthly payments, monthly rate,
        loan term, probability of default
        Output: expected loss expressed as a real
        number
    """
    t, Sp = survival_prob(pd, n)
    exp_loss = []
    S = np.insert(Sp, 0, 1)
    for k in np.arange(1, n + 1):
        PD_t = S[k - 1] - S[k]
        time, OB_t = outstanding_balance(R, i, k, n)
        exp_loss.append(PD_t * OB_t * (1 + i)**(-k) * 0.6)
    return sum(exp_loss)

EL_1 = expected_loss(
    ob1['monthly_payment'], ob1['monthly_rate'], ob1['loan_term_length'],
 ↪ob1['default_prob'])

EL_1
```

[58]:
```
1312.8065302973648
```

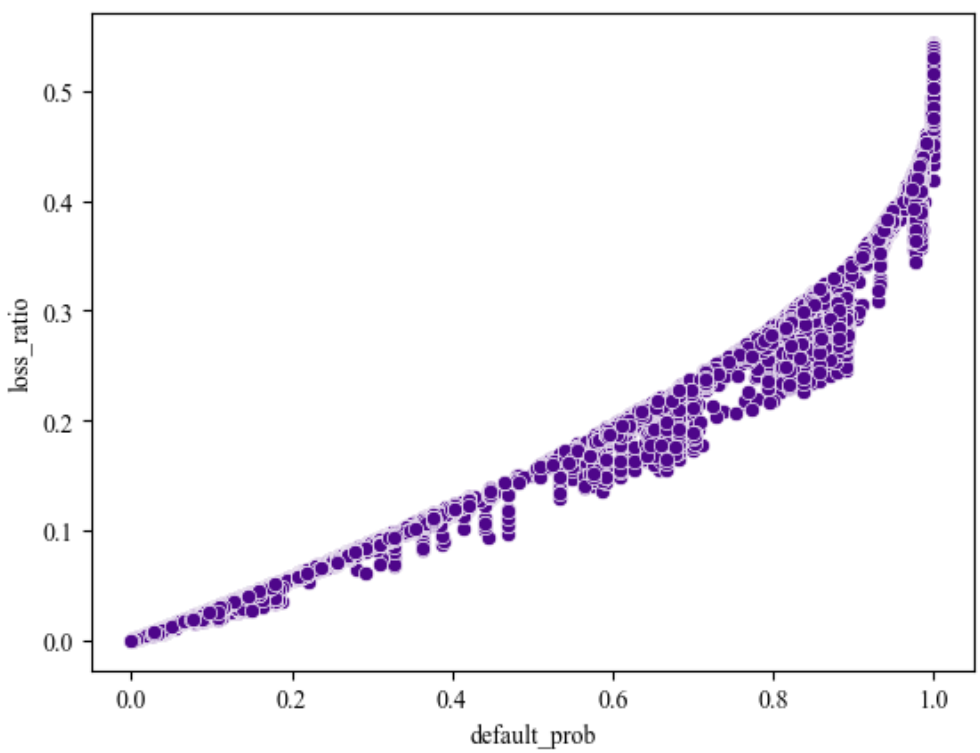
[59]:
```
# Adding expected loss column
ELoss = pd.DataFrame(np.zeros(len(df_amort)), columns = ['exp_loss'])

for i in range(len(df_amort)):
    ob_L = df_amort.iloc[i]
    ob_ELoss = expected_loss(
        ob_L['monthly_payment'], ob_L['monthly_rate'], ob_L['loan_term_length'],
 ↪ob_L['default_prob'])
    ELoss.iloc[i] = ob_ELoss

df_final = pd.concat([df_amort2, ELoss], axis = 1)
df_final[['loan_amnt', 'loan_int_rate', 'default_prob', 'loan_term_length',
 ↪'monthly_rate', 'monthly_payment', 'exp_loss']].sample(5)
```

[59]:
```
        loan_amnt  loan_int_rate  default_prob  loan_term_length  monthly_rate  \
12618     20000.0          15.27      0.856610                60      0.011913
2981       3200.0          14.79      0.999992                48      0.011561
16563      5000.0          10.99      0.000002                60      0.008727
```

```
18477    20000.0          6.54      0.999992                48      0.005293
30392    15000.0         11.71      0.983946                48      0.009271

       monthly_payment      exp_loss
12618       468.431304   6030.366708
2981         87.240995   1690.793390
16563       107.401213      0.002625
18477       472.933058  10722.340153
30392       388.592970   6484.967421
```

[60]:
```
df_final['loss_ratio'] = df_final['exp_loss']/df_final['loan_amnt']
sns.scatterplot(data = df_final, x = 'default_prob', y = 'loss_ratio', color =
 ↪'#4E068A')
max_ratio = df_final['loss_ratio'].max()
max_ratio
```

[60]: 0.5433088294062511



The convex shape of the loss to loan amount ratio is indicative of the survival probabilities decreasing as default probability increases, thereby increasing the expected loss. The maximum loss ratio is 54.33%, which is because of the Loss Given Default amount of 60%. For that reason, the loss ratio cannot possibly exceed 60%. The exponential nature of the loss ratio is apparent by looking at each tick mark: {(0, 0), (0.2, 0.05), (0.4, 0.1), (0.6, 0.15), (0.8, 0.25), (1, 0.5)}. So, while the growth is closely linear for default probabilities [0, 0.6], the risk accelerates onward, with lenders losing 40-55% of the initial loan amount for borrowers guaranteed to default.

8 Sensitivity

Many financial assets/derivatives are subject to volatility when interests rate change; loans are no exception. When interest rates change, loans are susceptible to refinancing or changes in the request amount itself. Thus, sensitivity is another key feature in credit-risk analysis.

For instance, if a lender wants to adjust the interest rate on a loan, it is useful to generate a spectrum of loan values under the new rate to match their needs. In this section, we will combine sensitivity and duration to investigate loan sensitivity under varying changes in the underlying interest rate.

The relevant computations include the Macaulay/Modified Duration. We will develop a function that uses a first-order Taylor Series approximation to estimate the updated value of a loan, P_{i_1} under a change in interest rate, Δi. Let ModD be the modified duration of the loan annuity payments, then

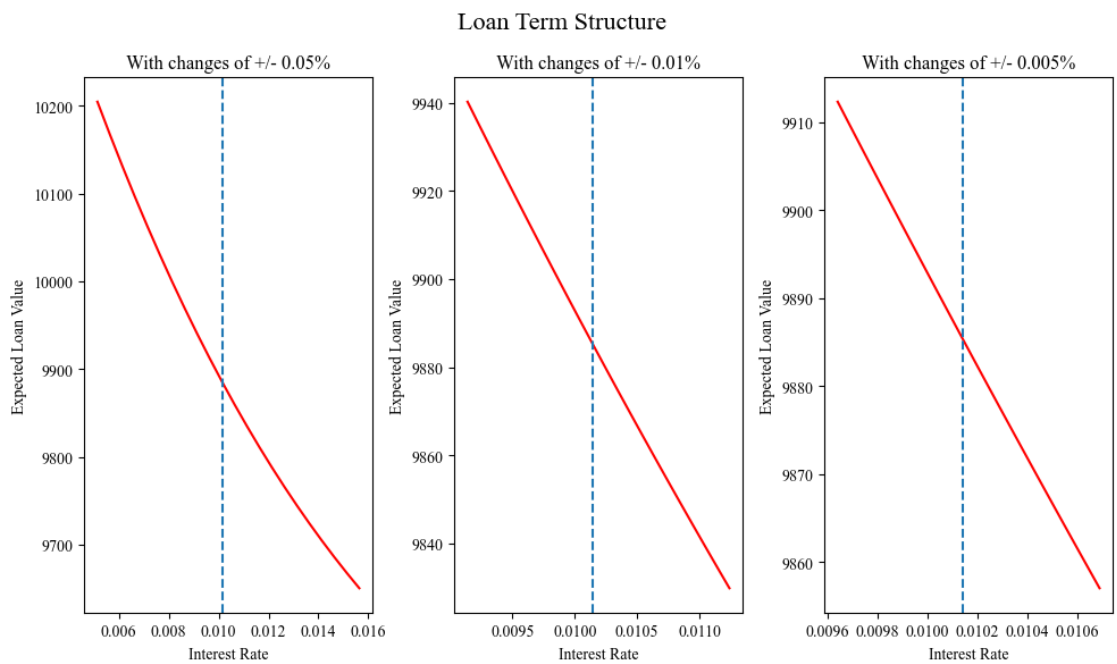
$$P_{i_1} = P_{i_0} - P_{i_0}(\Delta i)(\text{ModD})$$

This calculation will be used to assess updated loan values for changes in [-0.05%, 0.05%], [-0.01%, 0.01%], and [-0.0005%, 0.0005%], for a total of 21 interest rate observations.

[67]:
```
def MacD(i, n):
    """ returns the Macaulay Duration of an asset with
        level payments
    Arguments: monthly interest rate, loan term
    Output: Macaulay duration of the annuity, expressed
    as a positive number
    """
    v_i = (1 + i)**(-n)
    ann_imm = (1 - v_i)/i
    ann_due = ann_imm * (1 + i)
    ann_inc = (ann_due - n*v_i)/i
    macD = ann_inc/(12*ann_imm)
    return macD

def ModD(i, n):
    """ returns the Modified Duration of an asset
    Arguments: monthly interest rate, loan term
    Output: Modified duration of the annuity, expressed
    as a positive number
    """
    macD = MacD(i, n)
    modD = macD/(1 + i)
    return modD

def new_LV(L_i0, i, n, delta_i):
    """ returns the new loan value under changes in
    the interest rate
    Arguments: initial loan amount, monthly interest rate,
    loan term, change in interest rate (positive or negative)
    Output: the new loan value, expressed as a positive number
```

```
    """
    modD = ModD(i, n)
    L_i1 = L_i0 - (delta_i * L_i0 * modD)
    return L_i1

def LV_spectrum(L_i0, i, n, delta_i):
    """ outputs the spectrum of new values under changes in
    the interest rate
    Arguments: initial loan amount, monthly interest rate,
    loan term, change in interest rate (positive or negative)
    Output: a 21 x 2 numpy array containing the interest rate
    and its respective loan value
    """
    rate_arr = np.arange(i - delta_i, i + delta_i + delta_i/10, delta_i/10)
    loan_vals = []
    for k in rate_arr:
        macD = MacD(k, n)
        new_LV = L_i0 * ((1 + i)/(1 + k)) ** macD
        loan_vals.append(new_LV)

    return np.column_stack((rate_arr, loan_vals))

ob2 = df_amort2.sample().iloc[0]

# Plotting Loan Value Spectrum for a randomly selected borrower
fig, ax = plt.subplots(1, 3, figsize = (10, 6))

fig.suptitle('Loan Term Structure', fontsize = 16)

ob1_spec = LV_spectrum(
    ob2['exp_LV'], ob2['monthly_rate'], ob2['loan_term_length'], 0.005)

ax[0].plot(ob1_spec[:, 0], ob1_spec[:, 1], color = 'red')
ax[0].set_xlabel("Interest Rate")
ax[0].set_ylabel("Expected Loan Value")
ax[0].set_title("With changes of +/- 0.05%")
ax[0].axvline(x = ob2['monthly_rate'], linestyle = '--', linewidth = 1.5)

ob1_spec2 = LV_spectrum(
    ob2['exp_LV'], ob2['monthly_rate'], ob2['loan_term_length'], 0.001)

ax[1].plot(ob1_spec2[:, 0], ob1_spec2[:, 1], color = 'red')
ax[1].set_xlabel("Interest Rate")
ax[1].set_ylabel("Expected Loan Value")
ax[1].set_title("With changes of +/- 0.01%")
ax[1].axvline(x = ob2['monthly_rate'], linestyle = '--', linewidth = 1.5)
```

```
ob1_spec3 = LV_spectrum(
    ob2['exp_LV'], ob2['monthly_rate'], ob2['loan_term_length'], 0.0005)

ax[2].plot(ob1_spec3[:, 0], ob1_spec3[:, 1], color = 'red')
ax[2].set_xlabel("Interest Rate")
ax[2].set_ylabel("Expected Loan Value")
ax[2].set_title("With changes of +/- 0.005%")
ax[2].axvline(x = ob2['monthly_rate'], linestyle = '--', linewidth = 1.5)

plt.tight_layout()
plt.show()
```



The vertical line is used to represent the true monthly rate used in repaying the loan.

The loan term structure illustrates how the expected loan value responds to changes in the interest rate used for discounting. As the interest rate increases, the present value of expected payments declines due to heavier discounting. Even relatively small changes in the interest rate can produce noticeable changes in expected loan value due to compounding over the whole term. Note that at exceedingly small changes in the interest rate, the change in loan amount appears linear depsite the true relationship following a decaying pattern. This is due to linearization, or the idea that we can approximate values locally by taking the first-order Taylor expansion. When zooming close enough, the curvature is less visible and resembles a straight line, even if the underlying relationship remains nonlinear. Furthermore, the steep negative slope indicates that increases in interest rates substantially reduces the present value of expected loan payments, exposing the lender to interest rate risk.

9 Conclusion

Credit risk modeling helps us understand the potential dangers of approving loans to borrowers. When given demographic and historical data, we can apply machine learning and statistics techniques to minimize the chance of issuing false-positives and dampening the impact on lenders significantly. Credit risk modeling thrives from utilizing probabilistic modeling as a foundation; metrics like expected loan value and expected loss can exact how much lenders are giving up by approving loans, and sensitivities can map out amortization schedules under changes in interest rates. Overall, understanding how these methodologies are applied will help lenders make informed decisions on whether to follow through a loan and acknowledging the potential risks that come with accepting them.

Throughout this project, we have

- Identified early relationships between features and the target response. For example, there is potential multicollearity between age, employment experience, and credit history length. Also, we found that previous loan defaults appeared to have the strongest contribution in loan defaults.
- Recognized that type I errors are more economically catastrophic than type II errors from the lender's perspective.
- Found that a boosted ensemble model promises the highest predictablility and separability power, with F1-scores of 96% and 85% for default and paid-on-time, respectively, as well as an AUC of 97.93%.
- Designed an amortization schedule by assuming a loan term for each loan intent class, using a DTI ratio as a measure to decide whether a lender should approve a loan or not.
- Looked at interest and principal behavior and their influence on credit modeling.
- Used default probabilities given by the boosting model to measure dynamic credit risk metrics such as expected loan value, expected loss, and the lender's expected return.
- Analyzed the behavior between hazard rates, survival probabilities, expected return, and loss to loan amount ratio.
- Employed sensitivity modeling to expected loan values under small changes in the loan's monthly interest rate.

As such, there are always areas for improvement. Some of them are, but not limited to:

- Finding a more dynamic way of issuing loan terms for each borrower instead of fixing a term based on loan intent.
- Embellishing funding and recovery rates to more accurately reflect the lender's true expected return.
- Performing hyperparameter tuning in some machine learning models to improve the accuracy, F1-scores, and AUC.

10 References

Some of the information and code relating to machine learning techniques and packages were adapted from *An Introduction to Statistical Learning with Applications in Python* from Gareth M. James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor.

Unless otherwise stated, all of the outputs produced are my own.